

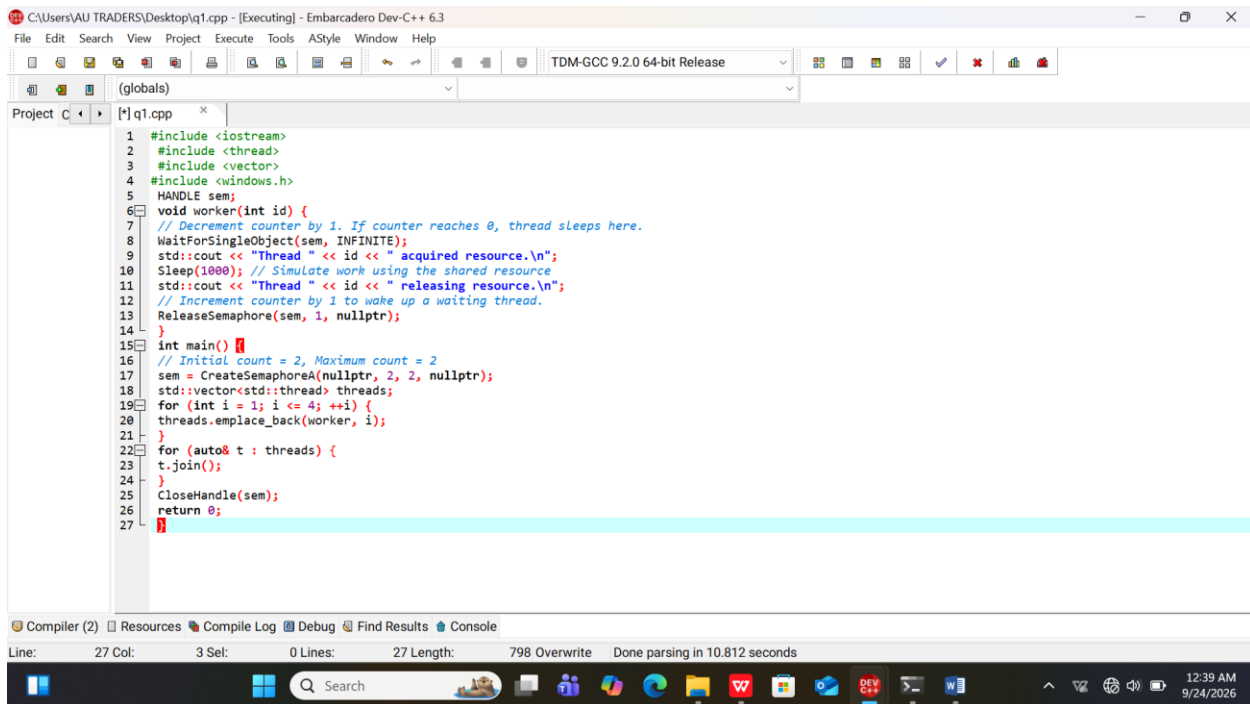
# **ASSIGNMENT**



|                     |                        |
|---------------------|------------------------|
| <b>Name</b>         | Alishba Nadeem         |
| <b>Roll Number</b>  | BCS-F24-E30            |
| <b>Assignment</b>   | Operating System       |
| <b>Teacher Name</b> | Sir Raza               |
| <b>Semester</b>     | 5 <sup>th</sup> (Self) |
| <b>Department</b>   | Computer Science       |

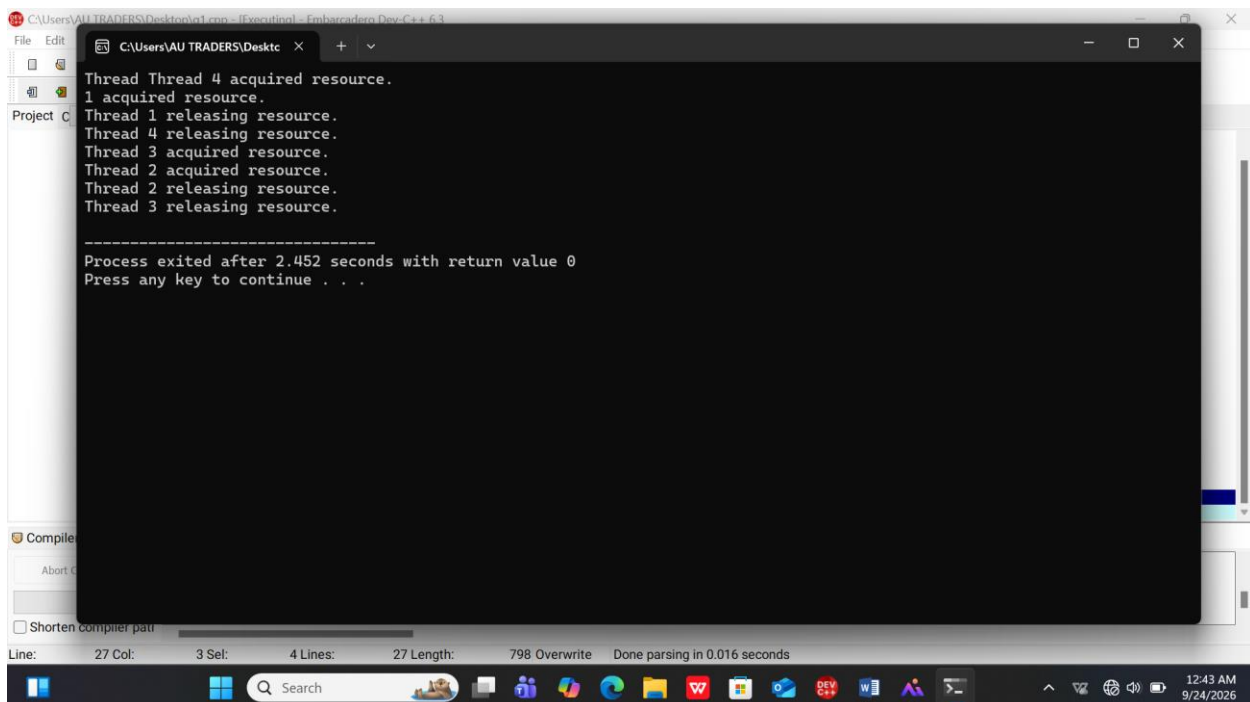
**University of Mianwali**

## Code 1:



```
1 #include <iostream>
2 #include <thread>
3 #include <vector>
4 #include <windows.h>
5 HANDLE sem;
6 void worker(int id) {
7     // Decrement counter by 1. If counter reaches 0, thread sleeps here.
8     WaitForSingleObject(sem, INFINITE);
9     std::cout << "Thread " << id << " acquired resource.\n";
10    Sleep(1000); // Simulate work using the shared resource
11    std::cout << "Thread " << id << " releasing resource.\n";
12    // Increment counter by 1 to wake up a waiting thread.
13    ReleaseSemaphore(sem, 1, nullptr);
14}
15 int main() {
16     // Initial count = 2, Maximum count = 2
17     sem = CreateSemaphoreA(nullptr, 2, 2, nullptr);
18     std::vector<std::thread> threads;
19     for (int i = 1; i <= 4; ++i) {
20         threads.emplace_back(worker, i);
21     }
22     for (auto& t : threads) {
23         t.join();
24     }
25     CloseHandle(sem);
26     return 0;
27 }
```

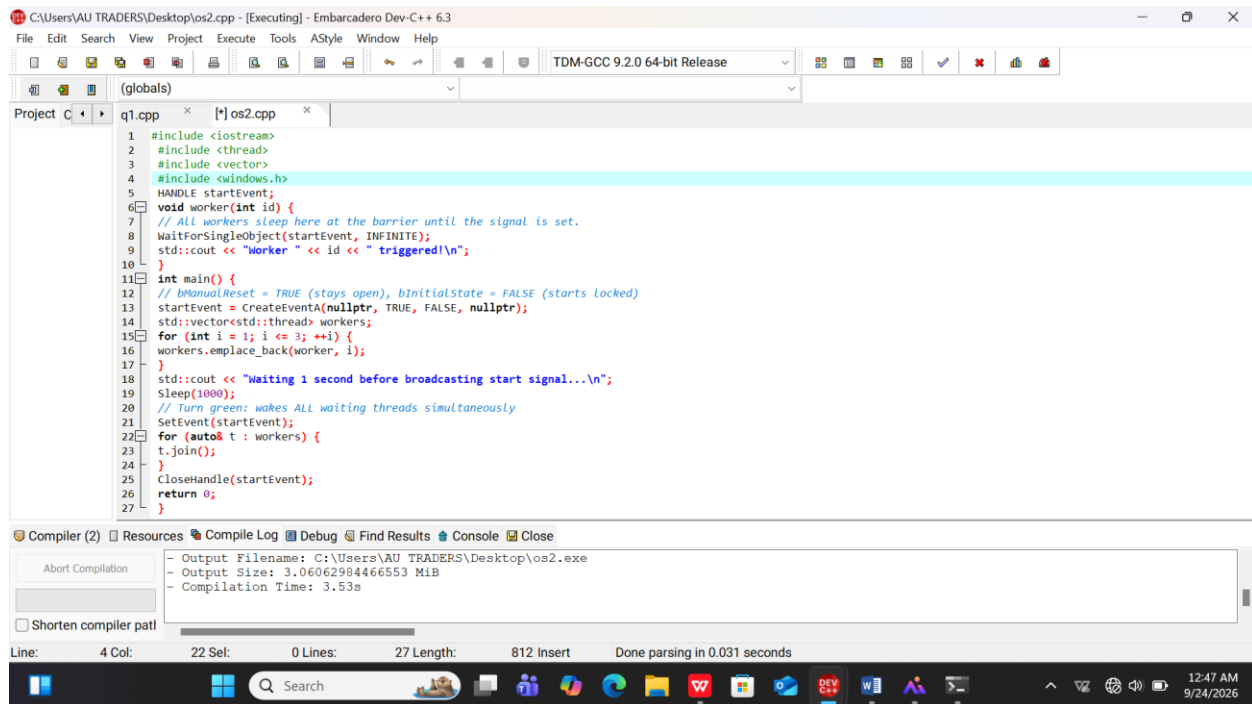
## Output:



```
Thread Thread 4 acquired resource.
1 acquired resource.
Thread 1 releasing resource.
Thread 4 releasing resource.
Thread 3 acquired resource.
Thread 2 acquired resource.
Thread 2 releasing resource.
Thread 3 releasing resource.

-----
Process exited after 2.452 seconds with return value 0
Press any key to continue . . .
```

## Code 2:



```
1 #include <iostream>
2 #include <thread>
3 #include <vector>
4 #include <windows.h>
5 HANDLE startEvent;
6 void worker(int id) {
7     // All workers sleep here at the barrier until the signal is set.
8     WaitForSingleObject(startEvent, INFINITE);
9     std::cout << "Worker " << id << " triggered!\n";
10 }
11
12 int main() {
13     // bManualReset = TRUE (stays open), bInitialState = FALSE (starts locked)
14     startEvent = CreateEventA(nullptr, TRUE, FALSE, nullptr);
15     std::vector<std::thread> workers;
16     for (int i = 1; i <= 3; ++i) {
17         workers.emplace_back(worker, i);
18     }
19     std::cout << "Waiting 1 second before broadcasting start signal...\n";
20     Sleep(1000);
21     // Turn green: wakes ALL waiting threads simultaneously
22     SetEvent(startEvent);
23     for (auto& t : workers) {
24         t.join();
25     }
26     CloseHandle(startEvent);
27     return 0;
28 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

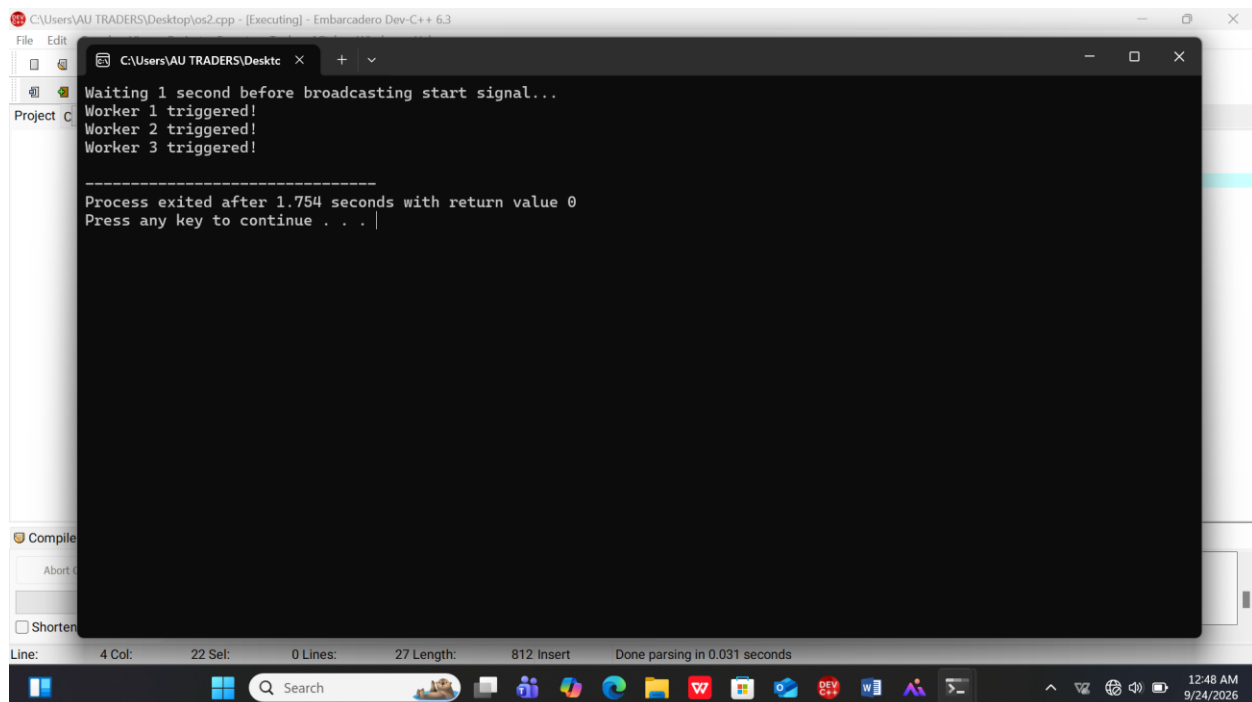
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os2.exe  
Output Size: 3.06062984466553 MiB  
Compilation Time: 3.53s

Line: 4 Col: 22 Sel: 0 Lines: 27 Length: 812 Insert Done parsing in 0.031 seconds

## Output:

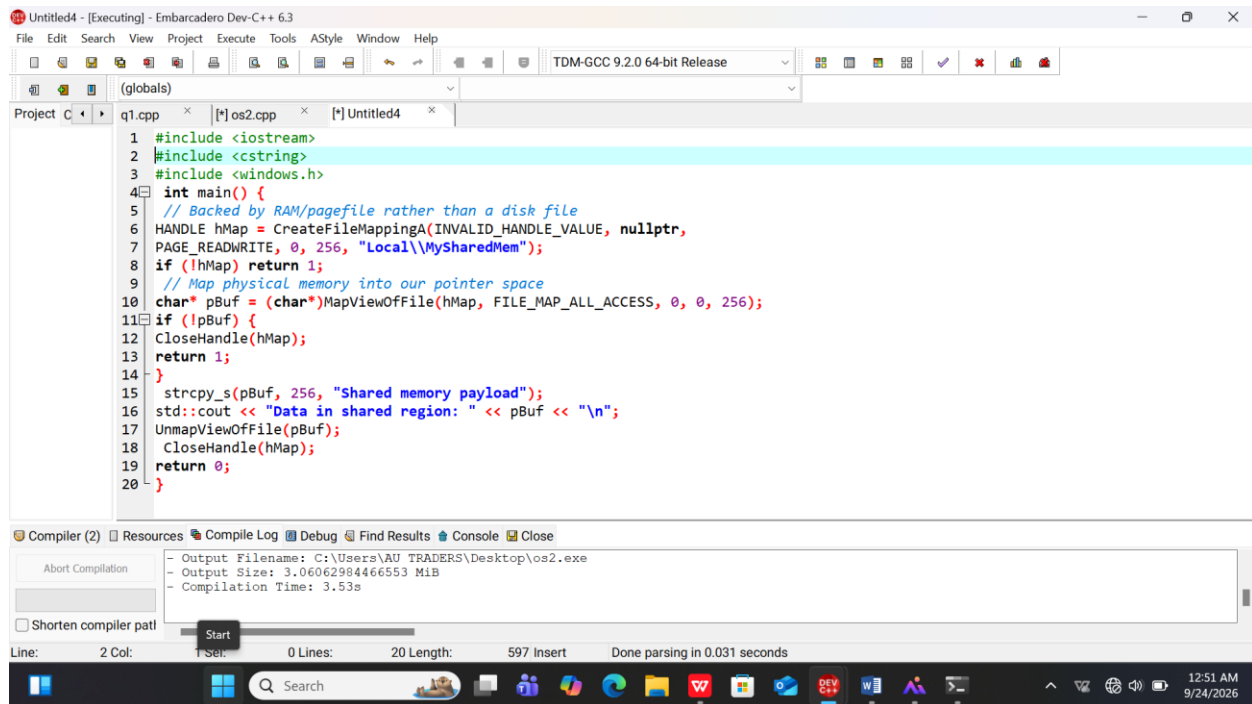


```
C:\Users\AU TRADERS\Desktop\os2.exe
Waiting 1 second before broadcasting start signal...
Worker 1 triggered!
Worker 2 triggered!
Worker 3 triggered!

-----
Process exited after 1.754 seconds with return value 0
Press any key to continue . . .
```

Line: 4 Col: 22 Sel: 0 Lines: 27 Length: 812 Insert Done parsing in 0.031 seconds

## Code 3:



```
1 #include <iostream>
2 #include <cstring>
3 #include <windows.h>
4 int main() {
5     // Backed by RAM/pagefile rather than a disk file
6     HANDLE hMap = CreateFileMappingA(INVALID_HANDLE_VALUE, nullptr,
7     PAGE_READWRITE, 0, 256, "Local\\MySharedMem");
8     if (!hMap) return 1;
9     // Map physical memory into our pointer space
10    char* pBuf = (char*)MapViewOfFile(hMap, FILE_MAP_ALL_ACCESS, 0, 0, 256);
11    if (!pBuf) {
12        CloseHandle(hMap);
13        return 1;
14    }
15    strcpy_s(pBuf, 256, "Shared memory payload");
16    std::cout << "Data in shared region: " << pBuf << "\n";
17    UnmapViewOfFile(pBuf);
18    CloseHandle(hMap);
19    return 0;
20 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

Abort Compilation

Shorten compiler path

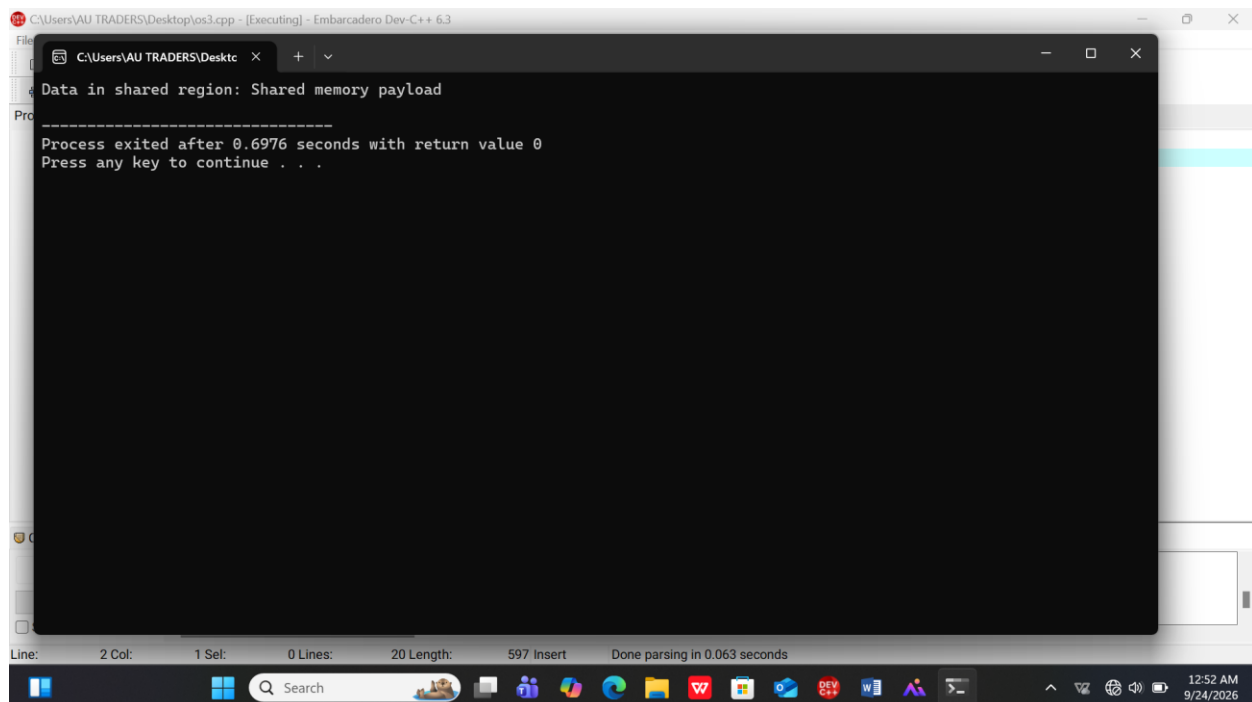
Start

Line: 2 Col: 2 Sel: 0 Lines: 20 Length: 597 Insert Done parsing in 0.031 seconds

Output Filename: C:\Users\AU TRADERS\Desktop\os2.exe  
Output Size: 3.06062984466553 MiB  
Compilation Time: 3.53s

12:51 AM 9/24/2026

## Output:

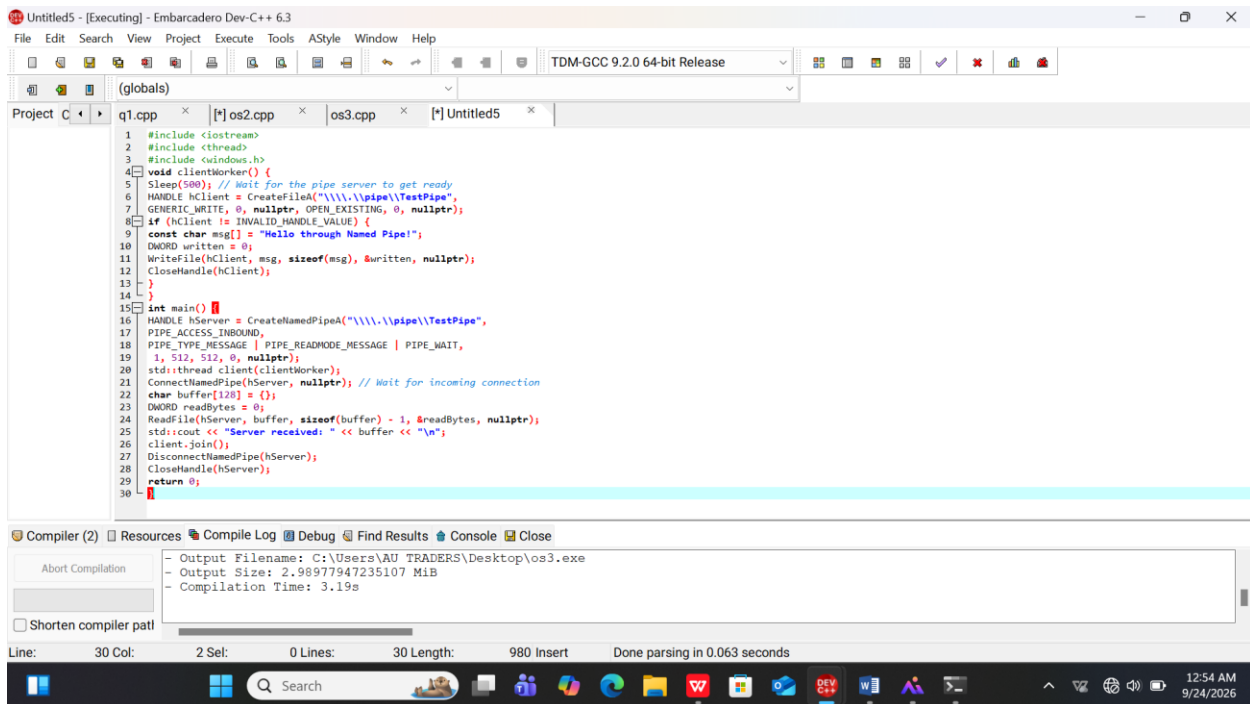


```
Data in shared region: Shared memory payload
-----
Process exited after 0.6976 seconds with return value 0
Press any key to continue . . .
```

Line: 2 Col: 2 Sel: 0 Lines: 20 Length: 597 Insert Done parsing in 0.063 seconds

12:52 AM 9/24/2026

## Code 4:



```
1 #include <iostream>
2 #include <thread>
3 #include <windows.h>
4 void clientWorker() {
5     Sleep(500); // Wait for the pipe server to get ready
6     HANDLE hClient = CreateFile(TEXT("\\\\.\\pipe\\TestPipe"),
7     GENERIC_WRITE, 0, nullptr, OPEN_EXISTING, 0, nullptr);
8     if (hClient != INVALID_HANDLE_VALUE) {
9         const char msg[] = "Hello through Named Pipe!";
10        DWORD written = 0;
11        WriteFile(hClient, msg, sizeof(msg), &written, nullptr);
12        CloseHandle(hClient);
13    }
14 }
15
16 int main() {
17     HANDLE hServer = CreateNamedPipeA(TEXT("\\\\.\\pipe\\TestPipe",
18     PIPE_ACCESS_INBOUND,
19     PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE | PIPE_WAIT,
20     1, 512, 0, nullptr);
21     std::thread client(clientWorker);
22     ConnectNamedPipe(hServer, nullptr); // Wait for incoming connection
23     char buffer[128] = {};
24     DWORD readBytes = 0;
25     ReadFile(hServer, buffer, sizeof(buffer) - 1, &readBytes, nullptr);
26     std::cout << "Server received: " << buffer << "\n";
27     client.join();
28     DisconnectNamedPipe(hServer);
29     CloseHandle(hServer);
30     return 0;
31 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

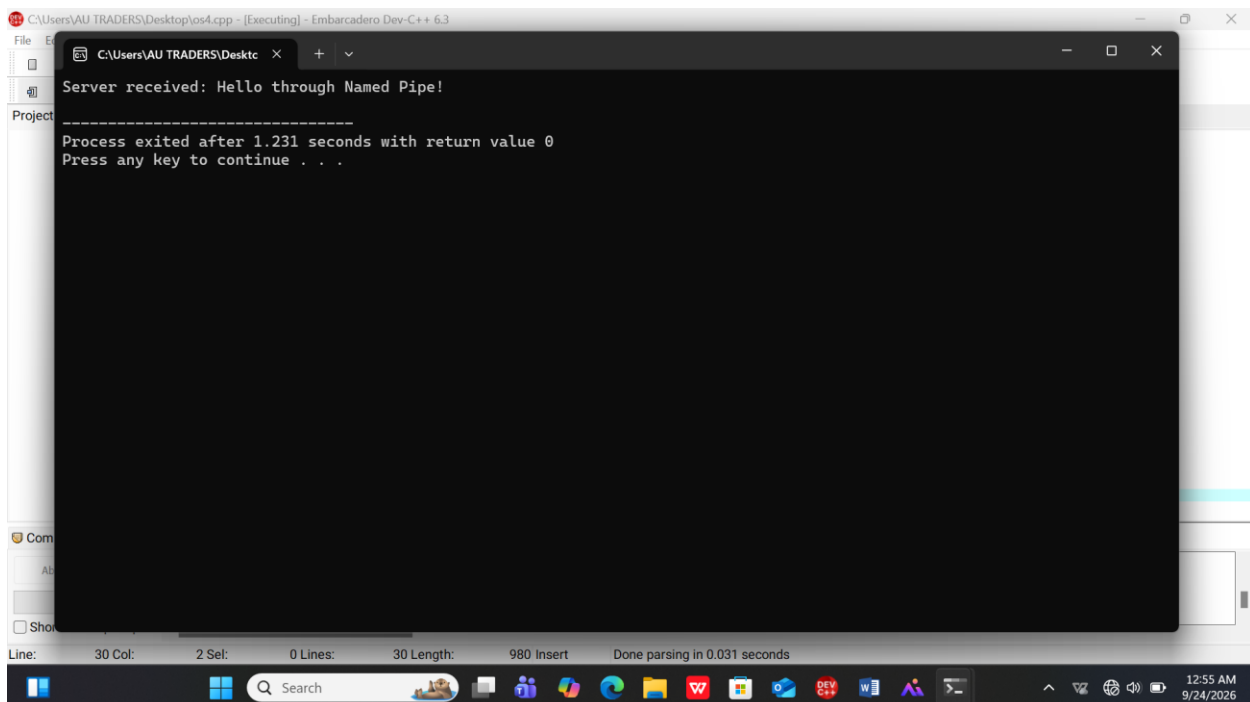
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os3.exe  
Output Size: 2.98977947235107 MiB  
Compilation Time: 3.19s

Line: 30 Col: 2 Sel: 0 Lines: 30 Length: 980 Insert Done parsing in 0.063 seconds

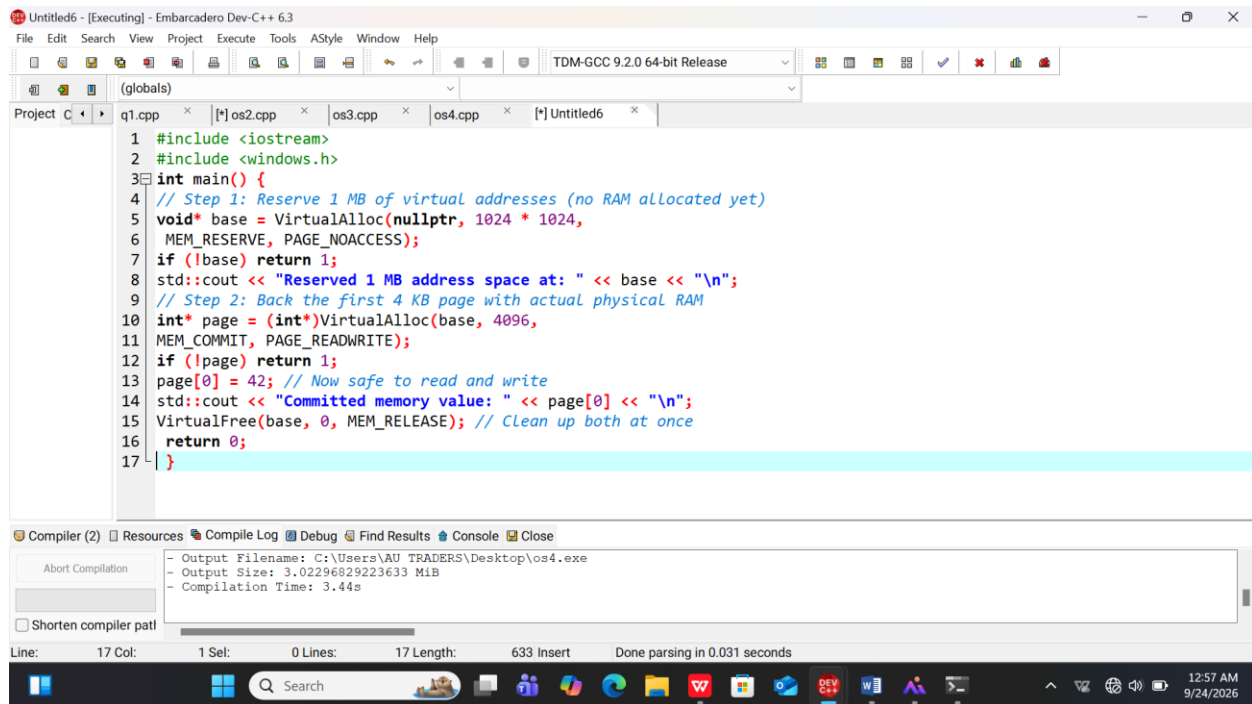
## Output:



```
Server received: Hello through Named Pipe!
-----
Process exited after 1.231 seconds with return value 0
Press any key to continue . . .
```

Line: 30 Col: 2 Sel: 0 Lines: 30 Length: 980 Insert Done parsing in 0.031 seconds

## Code 5:



```
1 #include <iostream>
2 #include <windows.h>
3 int main() {
4     // Step 1: Reserve 1 MB of virtual addresses (no RAM allocated yet)
5     void* base = VirtualAlloc(nullptr, 1024 * 1024,
6     MEM_RESERVE, PAGE_NOACCESS);
7     if (!base) return 1;
8     std::cout << "Reserved 1 MB address space at: " << base << "\n";
9     // Step 2: Back the first 4 KB page with actual physical RAM
10    int* page = (int*)VirtualAlloc(base, 4096,
11    MEM_COMMIT, PAGE_READWRITE);
12    if (!page) return 1;
13    page[0] = 42; // Now safe to read and write
14    std::cout << "Committed memory value: " << page[0] << "\n";
15    VirtualFree(base, 0, MEM_RELEASE); // Clean up both at once
16    return 0;
17 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

Abort Compilation

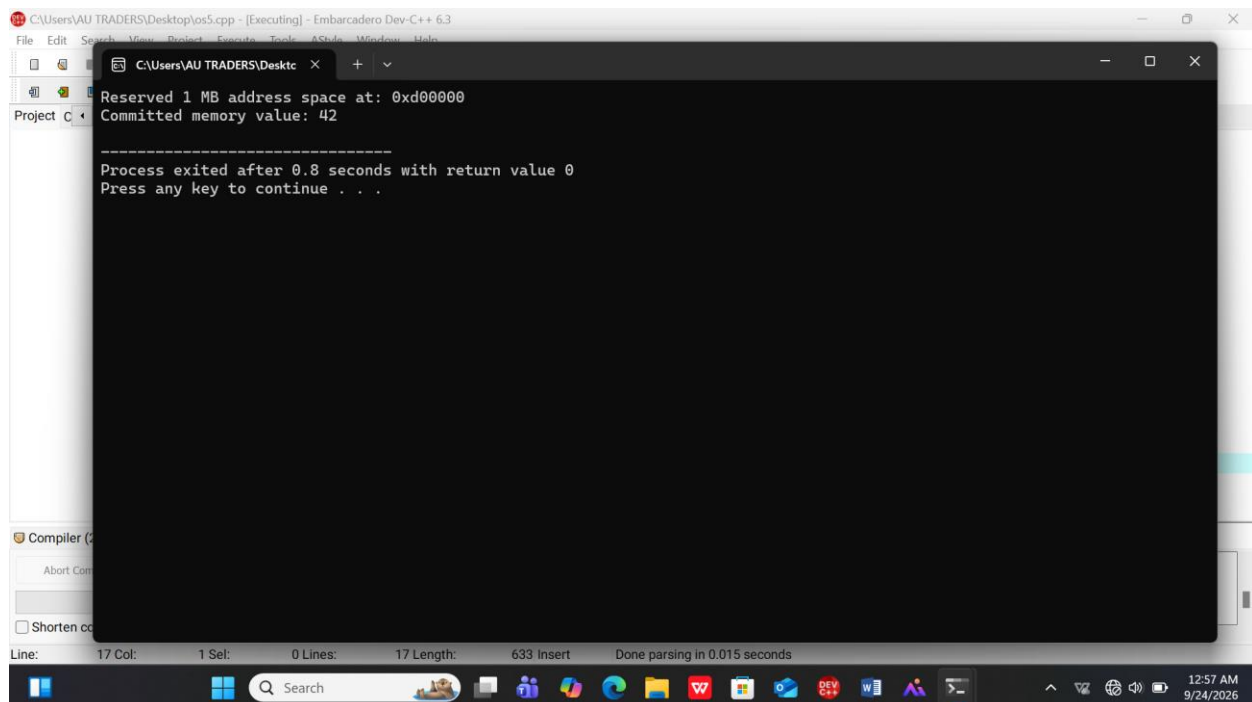
Shorten compiler path

Line: 17 Col: 1 Sel: 0 Lines: 17 Length: 633 Insert Done parsing in 0.031 seconds

Output Filename: C:\Users\AU TRADERS\Desktop\os4.exe  
Output Size: 3.02296829223633 MiB  
Compilation Time: 3.44s

12:57 AM 9/24/2026

## Output:



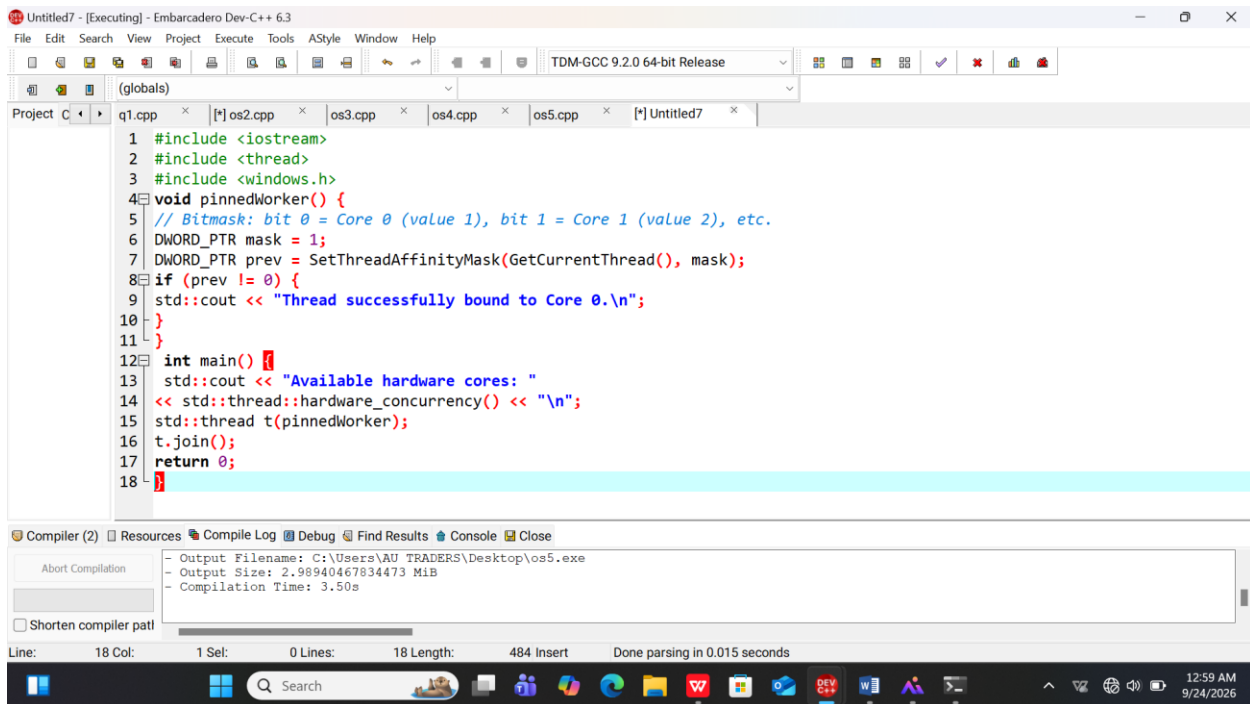
```
Reserved 1 MB address space at: 0xd00000
Committed memory value: 42

-----
Process exited after 0.8 seconds with return value 0
Press any key to continue . . .
```

Line: 17 Col: 1 Sel: 0 Lines: 17 Length: 633 Insert Done parsing in 0.015 seconds

12:57 AM 9/24/2026

## Code 6:



The screenshot shows the Embarcadero Dev-C++ IDE with the file 'os5.cpp' open. The code is as follows:

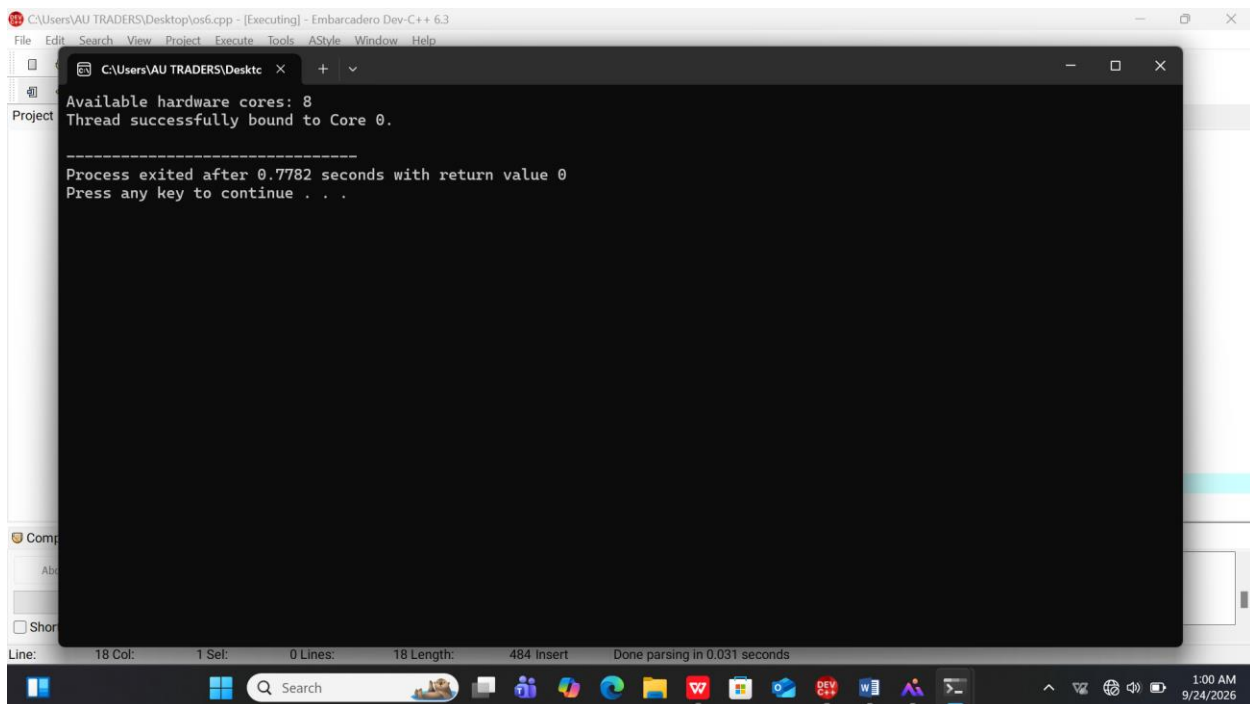
```
1 #include <iostream>
2 #include <thread>
3 #include <windows.h>
4 void pinnedWorker() {
5     // Bitmask: bit 0 = Core 0 (value 1), bit 1 = Core 1 (value 2), etc.
6     DWORD_PTR mask = 1;
7     DWORD_PTR prev = SetThreadAffinityMask(GetCurrentThread(), mask);
8     if (prev != 0) {
9         std::cout << "Thread successfully bound to Core 0.\n";
10    }
11 }
12 int main() {
13     std::cout << "Available hardware cores: "
14     << std::thread::hardware_concurrency() << "\n";
15     std::thread t(pinnedWorker);
16     t.join();
17     return 0;
18 }
```

Below the code editor, the 'Compiler (2)' window shows the following output:

```
- Output Filename: C:\Users\AU TRADERS\Desktop\os5.exe
- Output Size: 2.98940467834473 MiB
- Compilation Time: 3.50s
```

The status bar at the bottom indicates 'Line: 18 Col: 1 Sel: 0 Lines: 18 Length: 484 Insert Done parsing in 0.015 seconds'.

## Output:



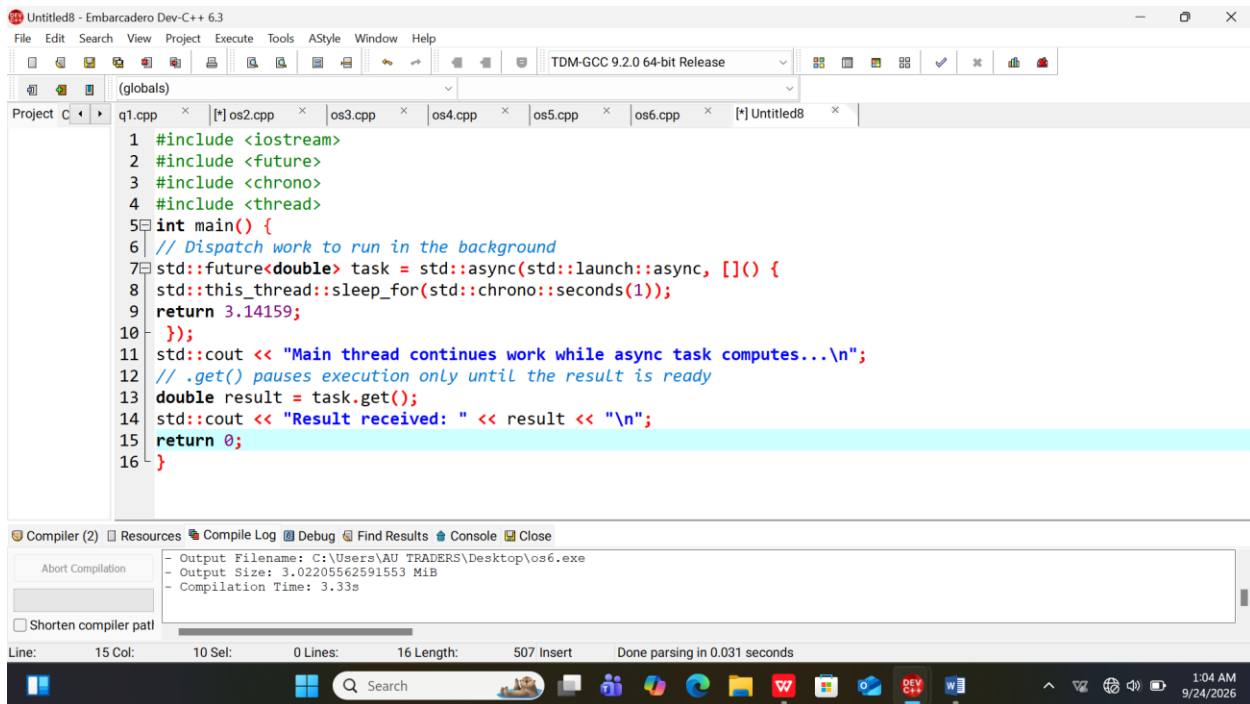
The screenshot shows the console output of the program. The text displayed is:

```
Available hardware cores: 8
Thread successfully bound to Core 0.

-----
Process exited after 0.7782 seconds with return value 0
Press any key to continue . . .
```

The status bar at the bottom indicates 'Line: 18 Col: 1 Sel: 0 Lines: 18 Length: 484 Insert Done parsing in 0.031 seconds'.

## Code 7:



```
1 #include <iostream>
2 #include <future>
3 #include <chrono>
4 #include <thread>
5 int main() {
6     // Dispatch work to run in the background
7     std::future<double> task = std::async(std::launch::async, []() {
8         std::this_thread::sleep_for(std::chrono::seconds(1));
9         return 3.14159;
10    });
11    std::cout << "Main thread continues work while async task computes...\n";
12    // .get() pauses execution only until the result is ready
13    double result = task.get();
14    std::cout << "Result received: " << result << "\n";
15    return 0;
16 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

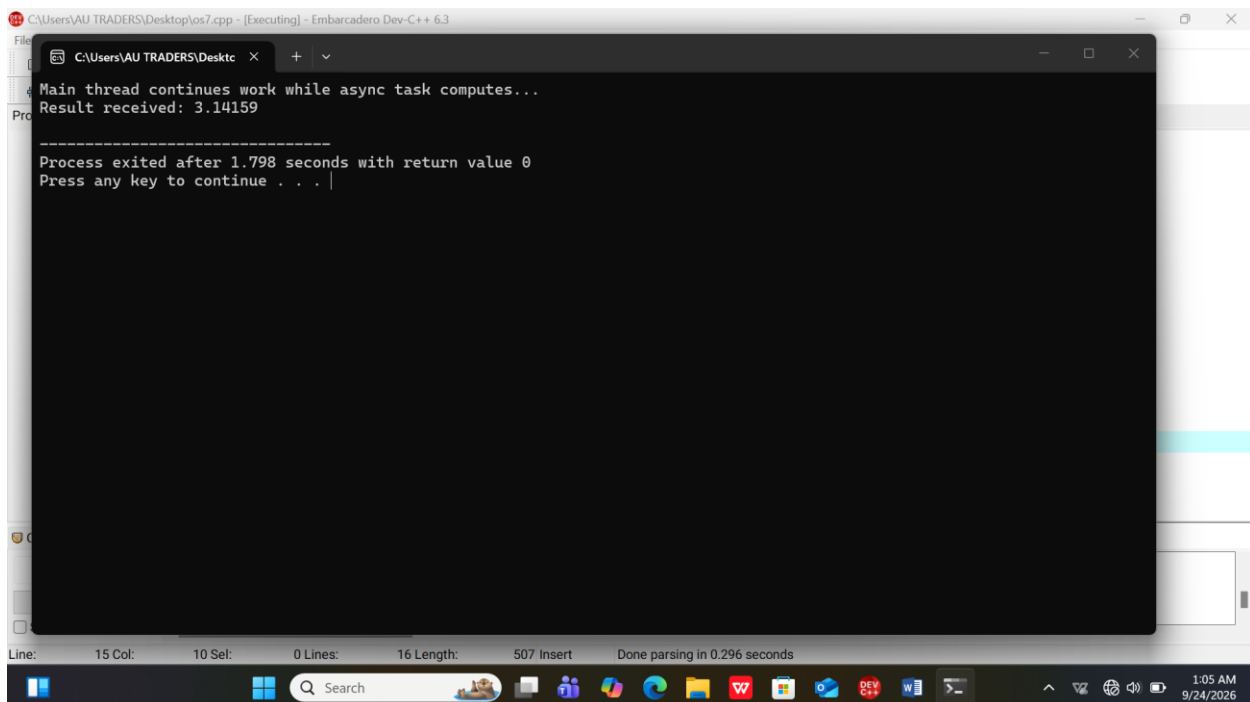
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os6.exe  
Output Size: 3.02205562591553 MiB  
Compilation Time: 3.33s

Line: 15 Col: 10 Sel: 0 Lines: 16 Length: 507 Insert Done parsing in 0.031 seconds

## Output:

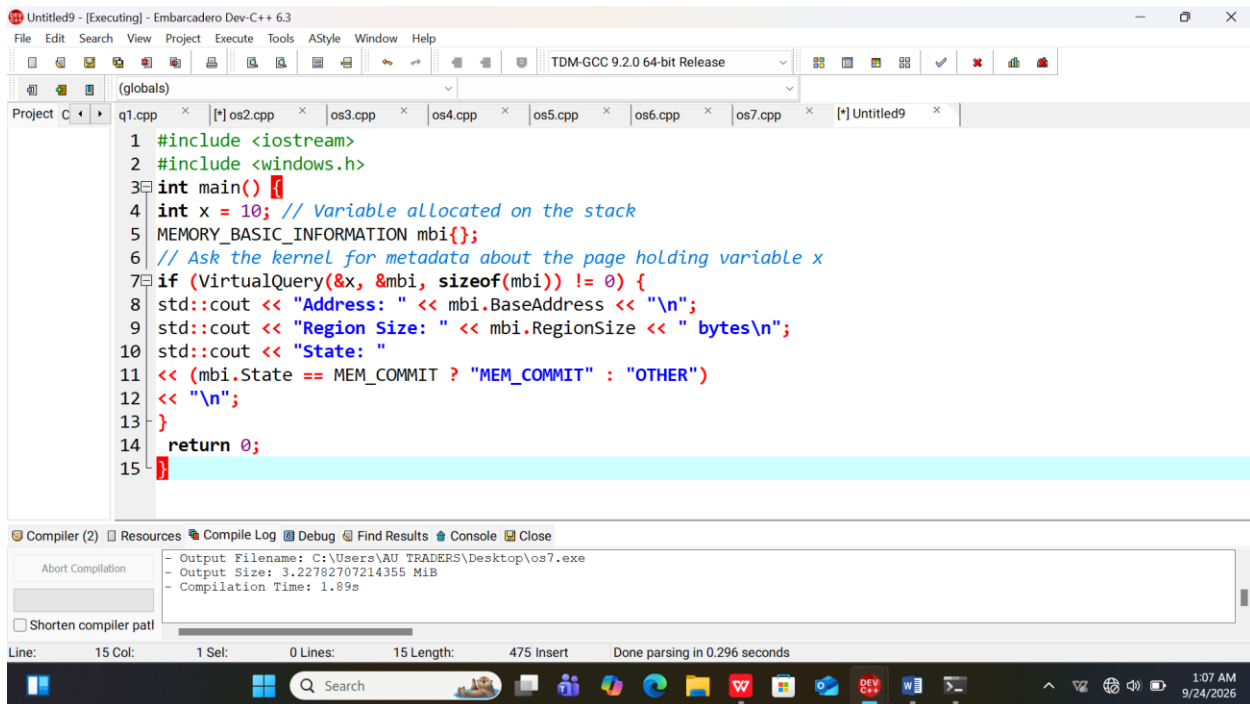


```
Main thread continues work while async task computes...
Result received: 3.14159

-----
Process exited after 1.798 seconds with return value 0
Press any key to continue . . .
```

Line: 15 Col: 10 Sel: 0 Lines: 16 Length: 507 Insert Done parsing in 0.296 seconds

## Code 8:



```
1 #include <iostream>
2 #include <windows.h>
3 int main() {
4     int x = 10; // Variable allocated on the stack
5     MEMORY_BASIC_INFORMATION mbi{};
6     // Ask the kernel for metadata about the page holding variable x
7     if (VirtualQuery(&x, &mbi, sizeof(mbi)) != 0) {
8         std::cout << "Address: " << mbi.BaseAddress << "\n";
9         std::cout << "Region Size: " << mbi.RegionSize << " bytes\n";
10        std::cout << "State: "
11        << (mbi.State == MEM_COMMIT ? "MEM_COMMIT" : "OTHER")
12        << "\n";
13    }
14    return 0;
15 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

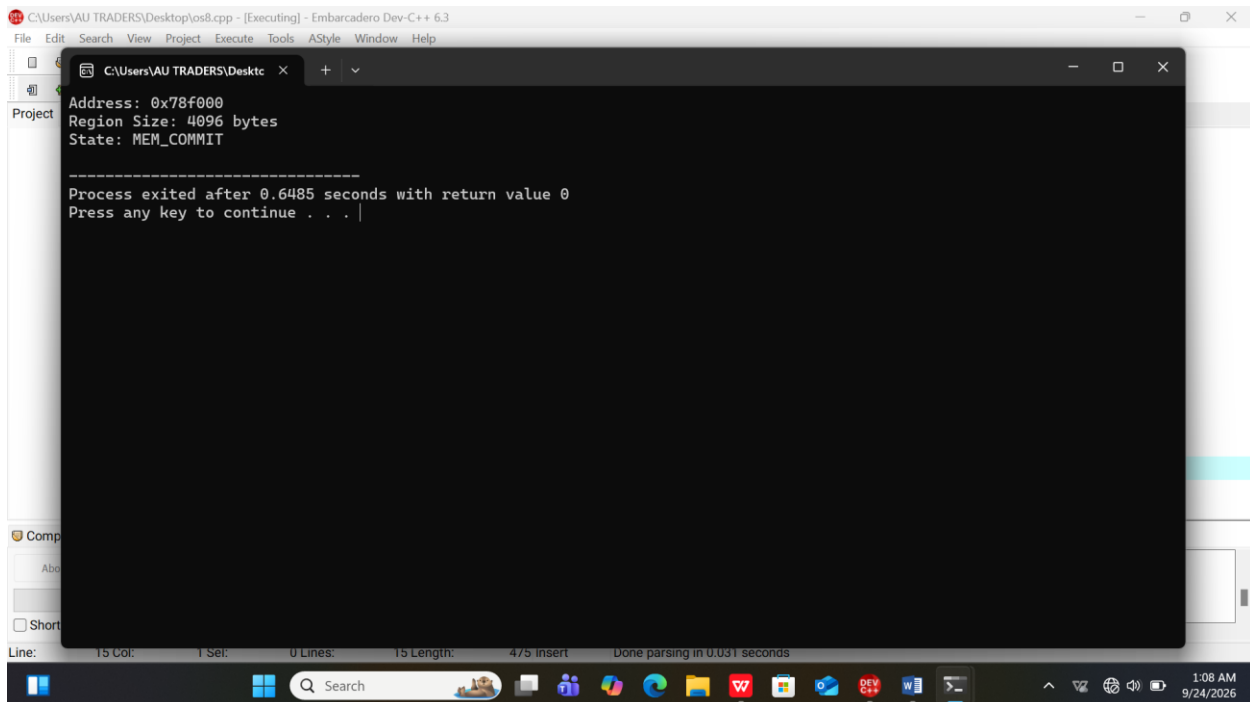
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os7.exe  
Output Size: 3.22782707214355 MiB  
Compilation Time: 1.89s

Line: 15 Col: 1 Sel: 0 Lines: 15 Length: 475 Insert Done parsing in 0.296 seconds

## Output:

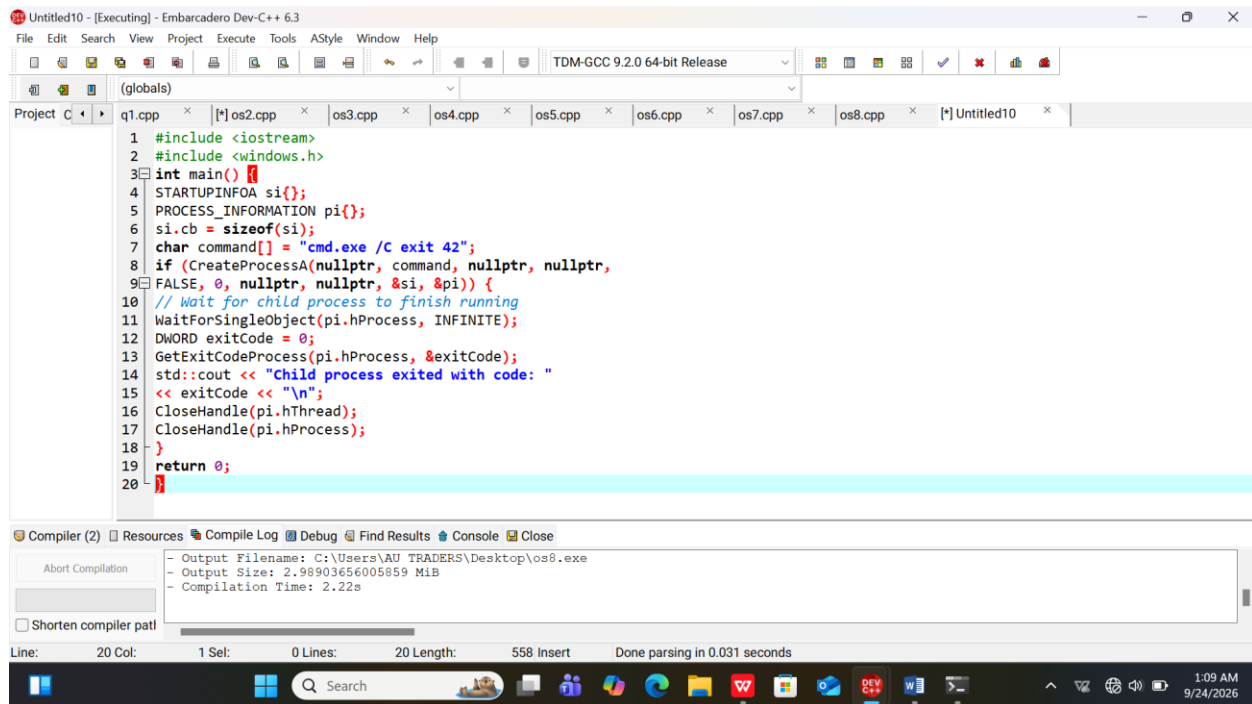


```
Address: 0x78f000
Region Size: 4096 bytes
State: MEM_COMMIT

-----
Process exited after 0.6485 seconds with return value 0
Press any key to continue . . .
```

Line: 15 Col: 1 Sel: 0 Lines: 15 Length: 475 Insert Done parsing in 0.031 seconds

## Code 9:



```
1 #include <iostream>
2 #include <windows.h>
3 int main() {
4     STARTUPINFOA si{};
5     PROCESS_INFORMATION pi{};
6     si.cb = sizeof(si);
7     char command[] = "cmd.exe /C exit 42";
8     if (CreateProcessA(nullptr, command, nullptr, nullptr,
9         FALSE, 0, nullptr, nullptr, &si, &pi)) {
10        // wait for child process to finish running
11        WaitForSingleObject(pi.hProcess, INFINITE);
12        DWORD exitCode = 0;
13        GetExitCodeProcess(pi.hProcess, &exitCode);
14        std::cout << "Child process exited with code: "
15        << exitCode << "\n";
16        CloseHandle(pi.hThread);
17        CloseHandle(pi.hProcess);
18    }
19    return 0;
20 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

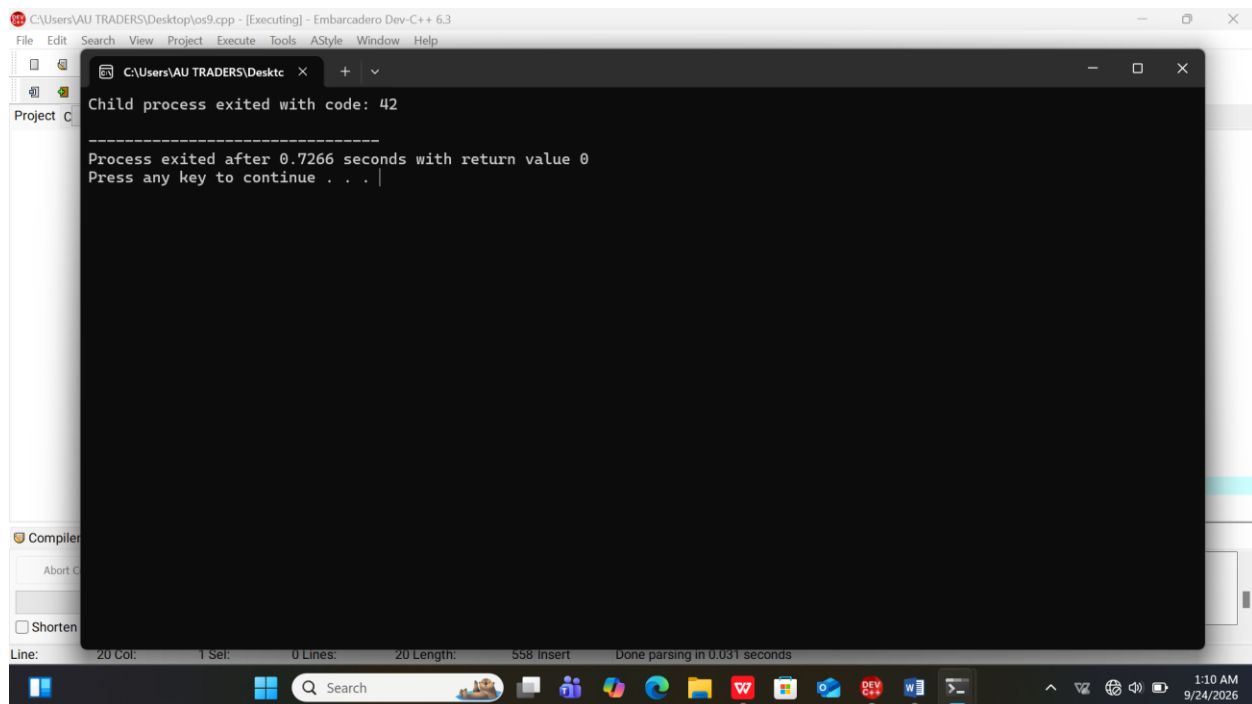
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os9.exe  
Output Size: 2.98903656005859 MiB  
Compilation Time: 2.22s

Line: 20 Col: 1 Sel: 0 Lines: 20 Length: 558 Insert Done parsing in 0.031 seconds

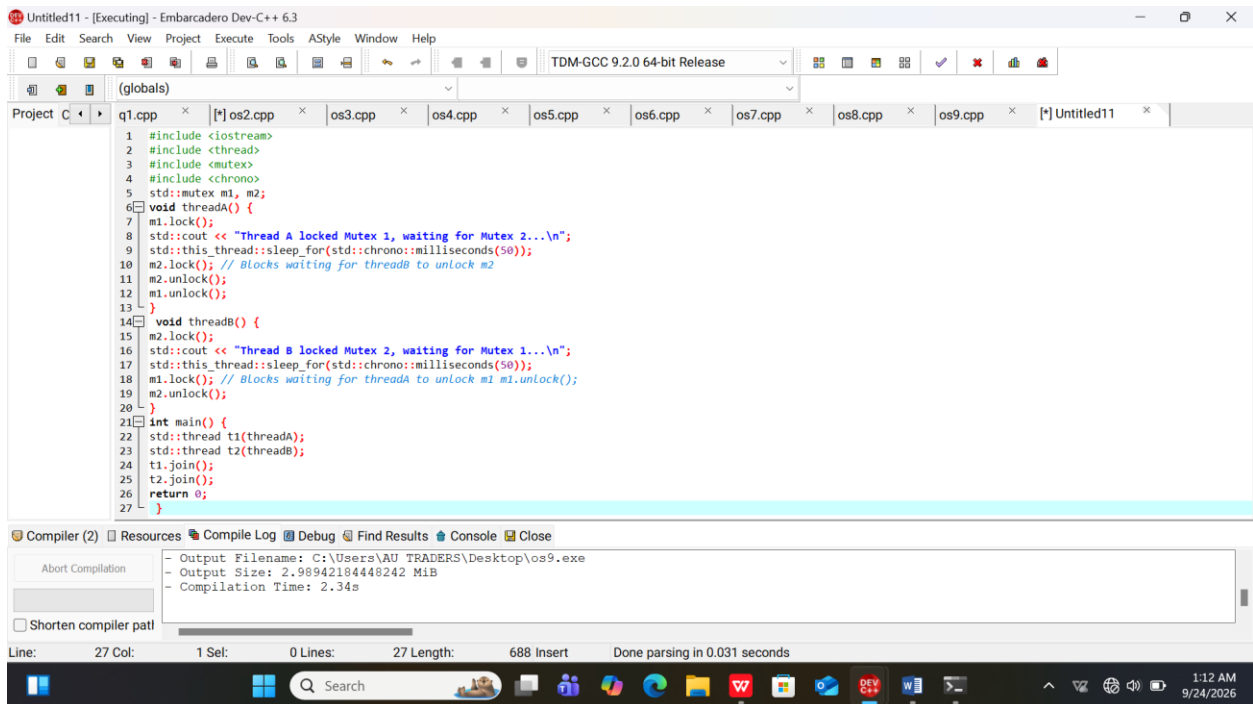
## Output:



```
Child process exited with code: 42
-----
Process exited after 0.7266 seconds with return value 0
Press any key to continue . . . |
```

Line: 20 Col: 1 Sel: 0 Lines: 20 Length: 558 Insert Done parsing in 0.031 seconds

## Code 10:



The screenshot shows an IDE window titled "Untitled11 - [Executing] - Embarcadero Dev-C++ 6.3". The code is in a file named "q1.cpp" and uses the "TDM-GCC 9.2.0 64-bit Release" compiler. The code implements two threads, threadA and threadB, which use two mutexes, m1 and m2, to perform a synchronized task. Thread A locks m1, prints a message, sleeps for 50ms, locks m2, prints another message, and then unlocks m2. Thread B locks m2, prints a message, sleeps for 50ms, locks m1, prints another message, and then unlocks m1. The main function creates both threads and joins them before returning 0.

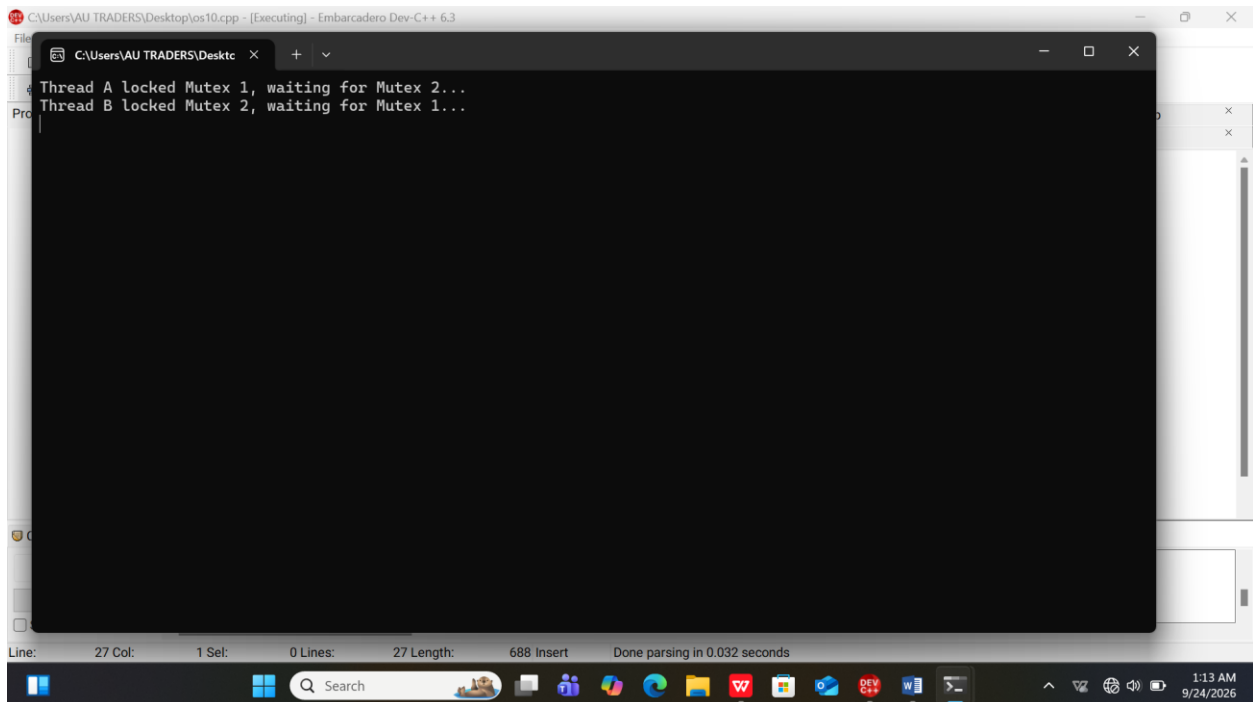
```
1 #include <iostream>
2 #include <thread>
3 #include <mutex>
4 #include <chrono>
5 std::mutex m1, m2;
6 void threadA() {
7     m1.lock();
8     std::cout << "Thread A locked Mutex 1, waiting for Mutex 2...\n";
9     std::this_thread::sleep_for(std::chrono::milliseconds(50));
10    m2.lock(); // Blocks waiting for threadB to unlock m2
11    m2.unlock();
12    m1.unlock();
13 }
14 void threadB() {
15     m2.lock();
16     std::cout << "Thread B locked Mutex 2, waiting for Mutex 1...\n";
17     std::this_thread::sleep_for(std::chrono::milliseconds(50));
18    m1.lock(); // Blocks waiting for threadA to unlock m1 m1.unlock();
19    m2.unlock();
20 }
21 int main() {
22     std::thread t1(threadA);
23     std::thread t2(threadB);
24     t1.join();
25     t2.join();
26     return 0;
27 }
```

The compiler output shows the following details:

- Output Filename: C:\Users\AU TRADERS\Desktop\os9.exe
- Output Size: 2.98942184448242 MiB
- Compilation Time: 2.34s

The status bar at the bottom indicates: Line: 27 Col: 1 Sel: 0 Lines: 27 Length: 688 Insert Done parsing in 0.031 seconds.

## Output:

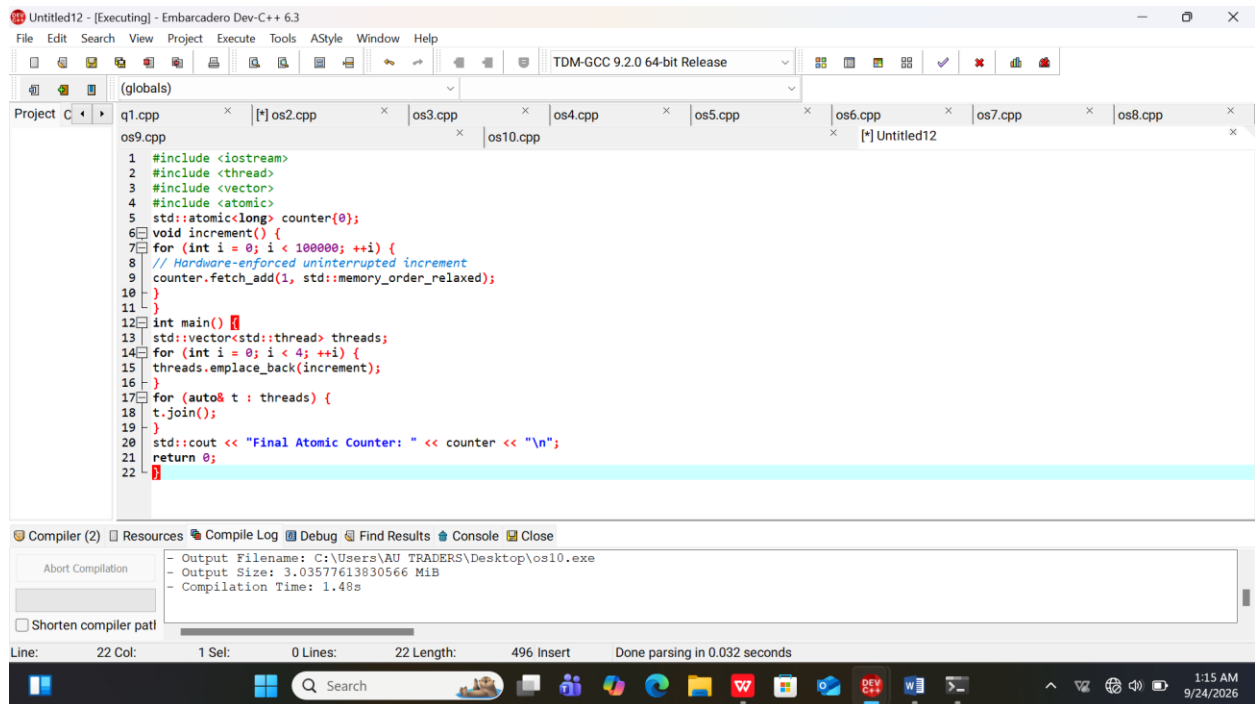


The screenshot shows a terminal window titled "C:\Users\AU TRADERS\Desktop\os10.cpp - [Executing] - Embarcadero Dev-C++ 6.3". The output of the program is displayed in the terminal:

```
Thread A locked Mutex 1, waiting for Mutex 2...
Thread B locked Mutex 2, waiting for Mutex 1...
```

The status bar at the bottom indicates: Line: 27 Col: 1 Sel: 0 Lines: 27 Length: 688 Insert Done parsing in 0.032 seconds.

## Code 11:



```
1 #include <iostream>
2 #include <thread>
3 #include <vector>
4 #include <atomic>
5 std::atomic<long> counter{0};
6 void increment() {
7     for (int i = 0; i < 10000; ++i) {
8         // Hardware-enforced uninterruptible increment
9         counter.fetch_add(1, std::memory_order_relaxed);
10    }
11 }
12 int main() {
13     std::vector<std::thread> threads;
14     for (int i = 0; i < 4; ++i) {
15         threads.emplace_back(increment);
16     }
17     for (auto& t : threads) {
18         t.join();
19     }
20     std::cout << "Final Atomic Counter: " << counter << "\n";
21     return 0;
22 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

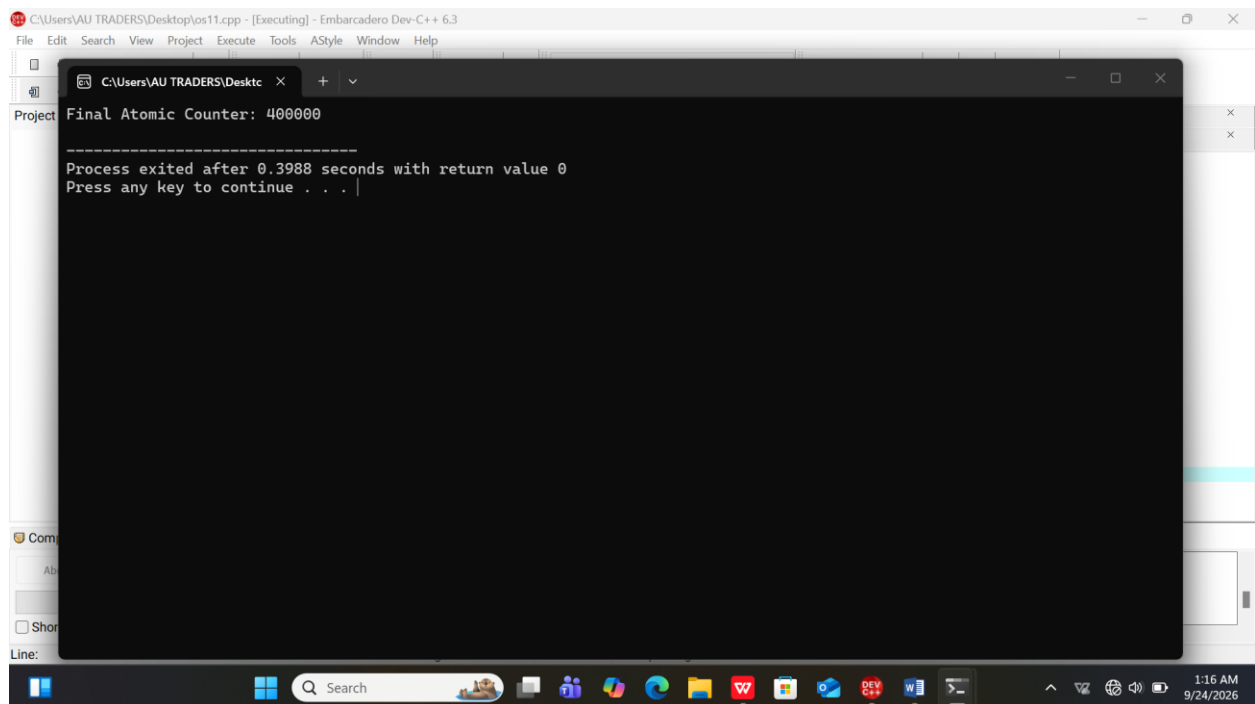
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os10.exe  
Output Size: 3.03577613830566 MiB  
Compilation Time: 1.48s

Line: 22 Col: 1 Sel: 0 Lines: 22 Length: 496 Insert Done parsing in 0.032 seconds

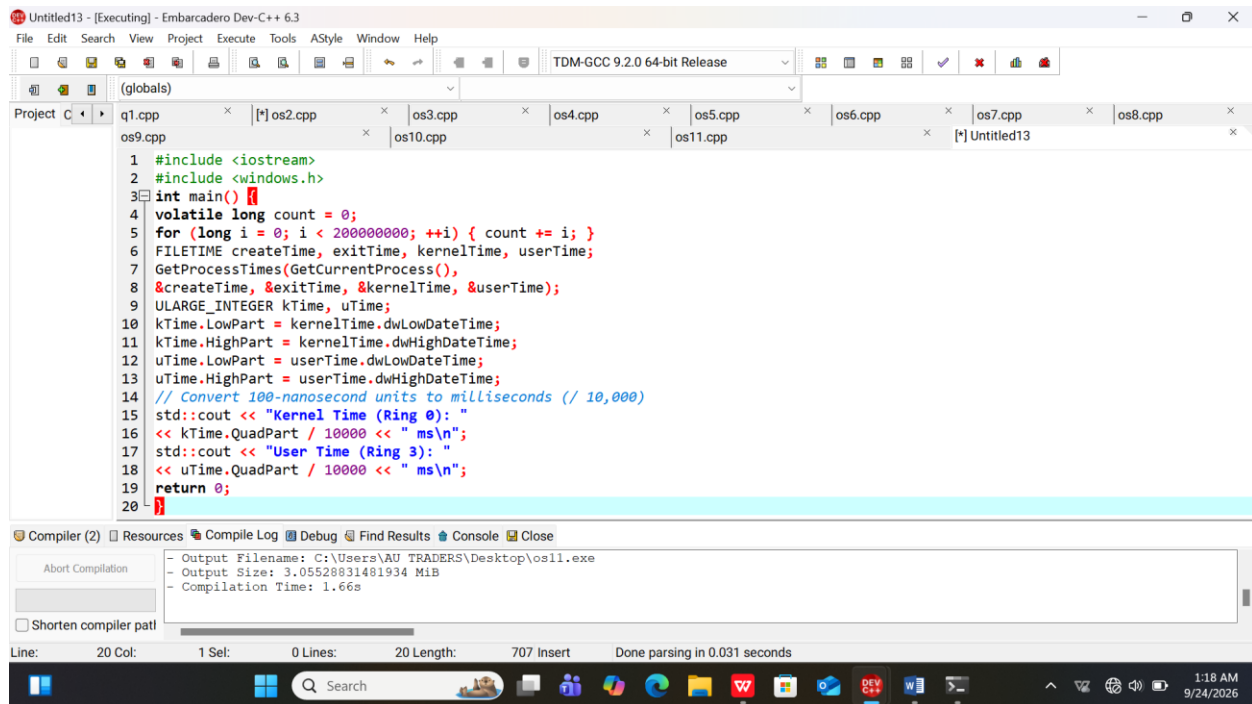
## Output:



```
Final Atomic Counter: 40000

-----
Process exited after 0.3988 seconds with return value 0
Press any key to continue . . .
```

## Code 12:



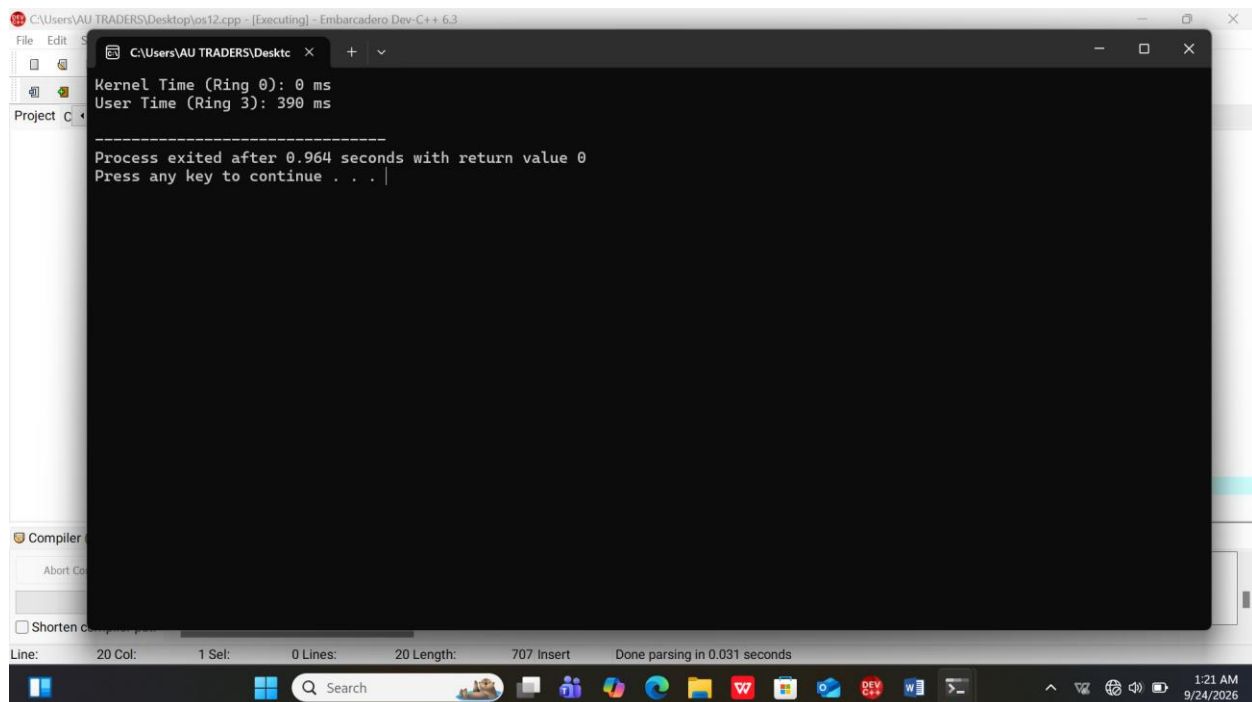
```
1 #include <iostream>
2 #include <windows.h>
3 int main()
4 {
5     volatile long count = 0;
6     for (long i = 0; i < 200000000; ++i) { count += i; }
7     FILETIME createTime, exitTime, kernelTime, userTime;
8     GetProcessTimes(GetCurrentProcess(),
9     &createTime, &exitTime, &kernelTime, &userTime);
10    ULARGE_INTEGER kTime, uTime;
11    kTime.LowPart = kernelTime.dwLowDateTime;
12    kTime.HighPart = kernelTime.dwHighDateTime;
13    uTime.LowPart = userTime.dwLowDateTime;
14    uTime.HighPart = userTime.dwHighDateTime;
15    // Convert 100-nanosecond units to milliseconds (/ 10,000)
16    std::cout << "Kernel Time (Ring 0): "
17    << kTime.QuadPart / 10000 << " ms\n";
18    std::cout << "User Time (Ring 3): "
19    << uTime.QuadPart / 10000 << " ms\n";
20    return 0;
21 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

Output Filename: C:\Users\AU TRADERS\Desktop\os11.exe  
Output Size: 3.0528831481934 MiB  
Compilation Time: 1.66s

Line: 20 Col: 1 Sel: 0 Lines: 20 Length: 707 Insert Done parsing in 0.031 seconds

## Output:

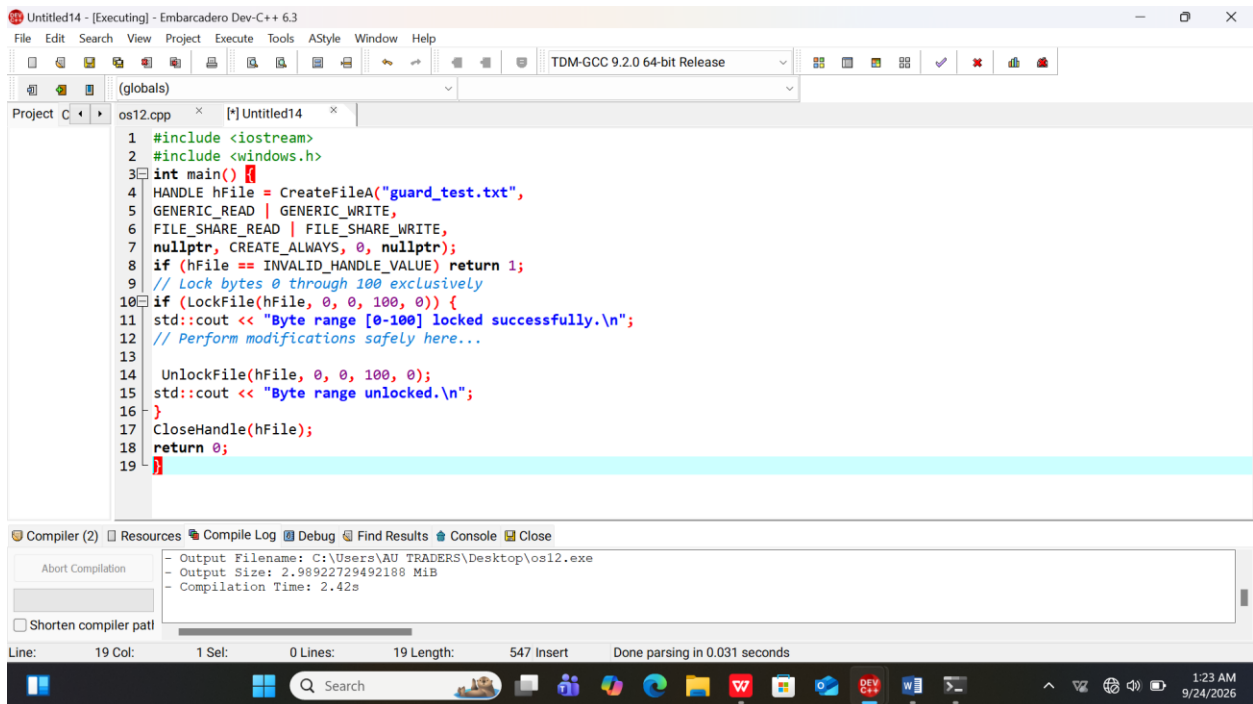


```
Kernel Time (Ring 0): 0 ms
User Time (Ring 3): 390 ms

-----
Process exited after 0.964 seconds with return value 0
Press any key to continue . . .
```

Line: 20 Col: 1 Sel: 0 Lines: 20 Length: 707 Insert Done parsing in 0.031 seconds

## Code 13:



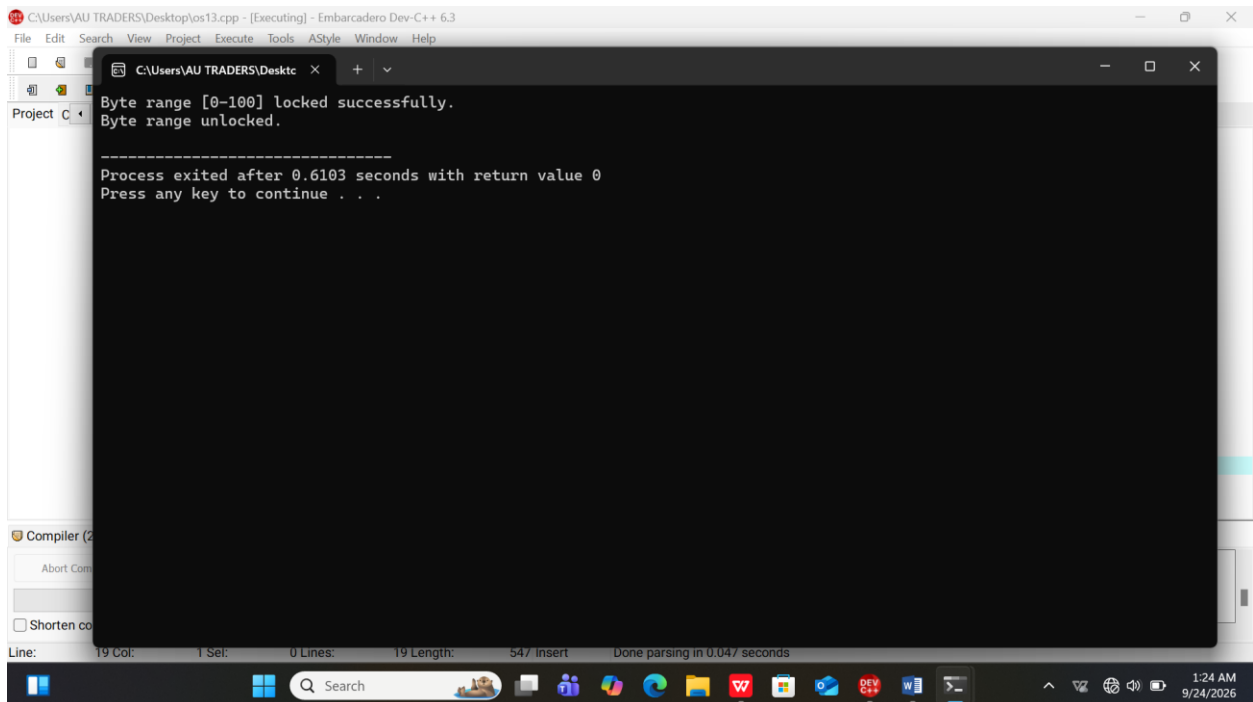
```
1 #include <iostream>
2 #include <windows.h>
3 int main() {
4     HANDLE hFile = CreateFileA("guard_test.txt",
5     GENERIC_READ | GENERIC_WRITE,
6     FILE_SHARE_READ | FILE_SHARE_WRITE,
7     nullptr, CREATE_ALWAYS, 0, nullptr);
8     if (hFile == INVALID_HANDLE_VALUE) return 1;
9     // Lock bytes 0 through 100 exclusively
10    if (LockFile(hFile, 0, 0, 100, 0)) {
11        std::cout << "Byte range [0-100] locked successfully.\n";
12        // Perform modifications safely here...
13    }
14    UnlockFile(hFile, 0, 0, 100, 0);
15    std::cout << "Byte range unlocked.\n";
16 }
17 CloseHandle(hFile);
18 return 0;
19 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

Output Filename: C:\Users\AU TRADERS\Desktop\os12.exe  
Output Size: 2.98922729492188 MiB  
Compilation Time: 2.42s

Line: 19 Col: 1 Sel: 0 Lines: 19 Length: 547 Insert Done parsing in 0.031 seconds

## Output:

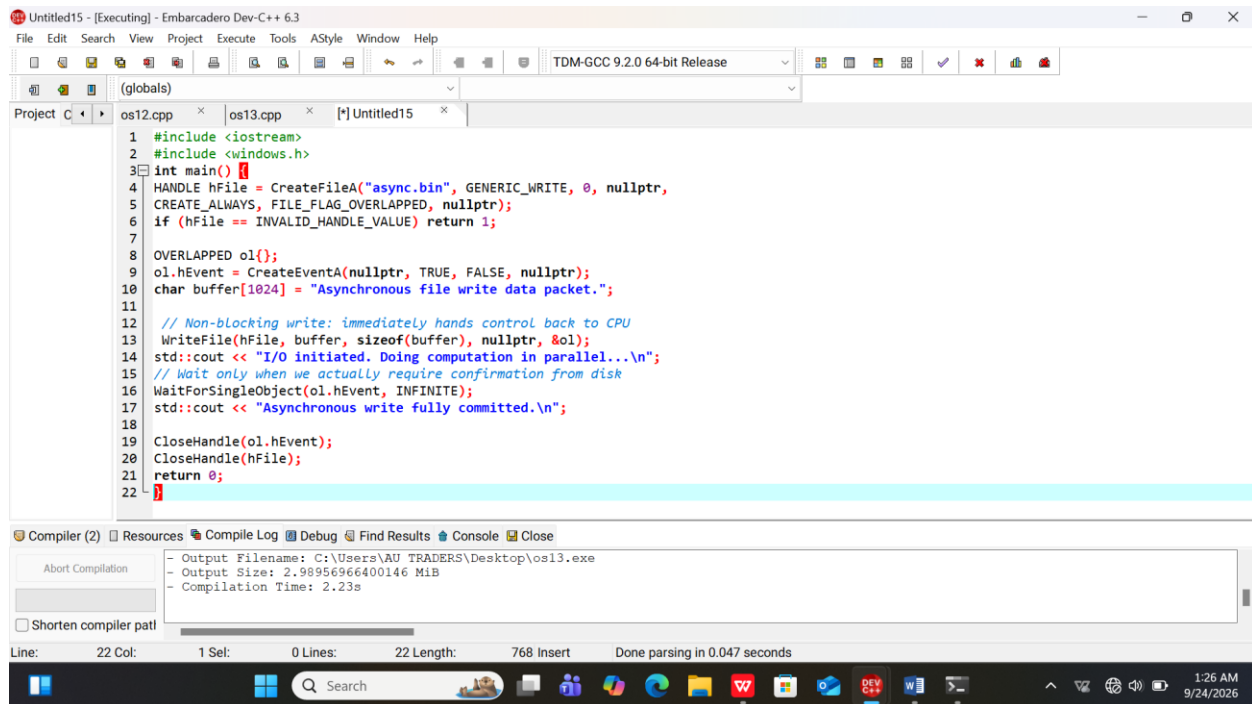


```
Byte range [0-100] locked successfully.
Byte range unlocked.

-----
Process exited after 0.6103 seconds with return value 0
Press any key to continue . . .
```

Line: 19 Col: 1 Sel: 0 Lines: 19 Length: 547 Insert Done parsing in 0.047 seconds

## Code 14:



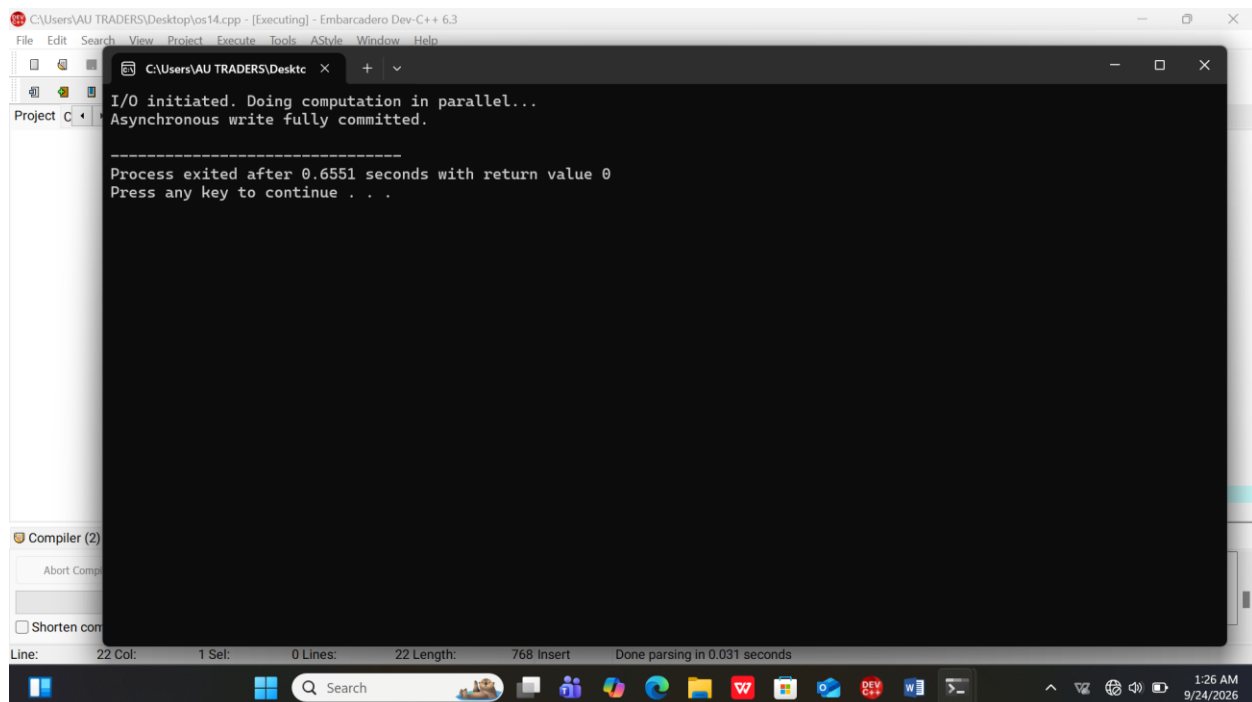
```
1 #include <iostream>
2 #include <windows.h>
3 int main()
4 {
5     HANDLE hFile = CreateFileA("async.bin", GENERIC_WRITE, 0, nullptr,
6     CREATE_ALWAYS, FILE_FLAG_OVERLAPPED, nullptr);
7     if (hFile == INVALID_HANDLE_VALUE) return 1;
8
9     OVERLAPPED ol{};
10    ol.hEvent = CreateEventA(nullptr, TRUE, FALSE, nullptr);
11    char buffer[1024] = "Asynchronous file write data packet.";
12
13    // Non-blocking write: immediately hands control back to CPU
14    WriteFile(hFile, buffer, sizeof(buffer), nullptr, &ol);
15    std::cout << "I/O initiated. Doing computation in parallel...\n";
16    // Wait only when we actually require confirmation from disk
17    WaitForSingleObject(ol.hEvent, INFINITE);
18    std::cout << "Asynchronous write fully committed.\n";
19
20    CloseHandle(ol.hEvent);
21    CloseHandle(hFile);
22    return 0;
23 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

Output Filename: C:\Users\AU TRADERS\Desktop\os13.exe  
Output Size: 2.98956966400146 MiB  
Compilation Time: 2.23s

Line: 22 Col: 1 Sel: 0 Lines: 22 Length: 768 Insert Done parsing in 0.047 seconds

## Output:

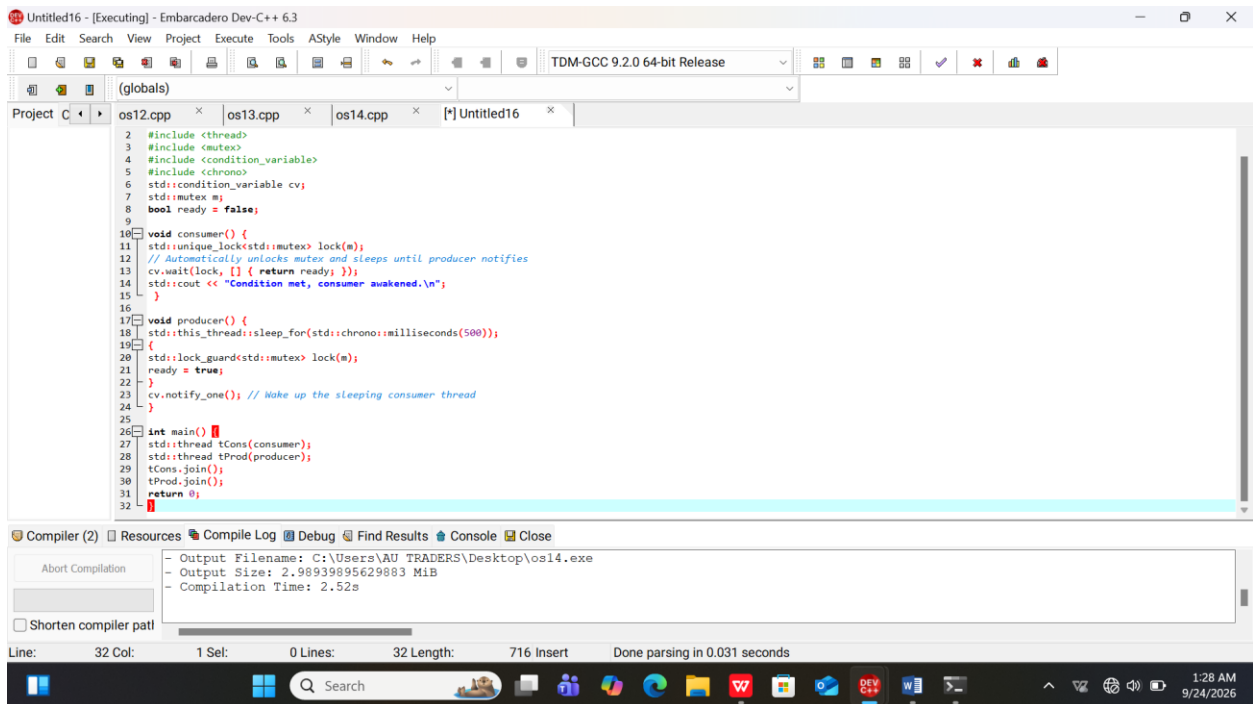


```
I/O initiated. Doing computation in parallel...
Asynchronous write fully committed.

-----
Process exited after 0.6551 seconds with return value 0
Press any key to continue . . .
```

Line: 22 Col: 1 Sel: 0 Lines: 22 Length: 768 Insert Done parsing in 0.031 seconds

## Code 15:



The screenshot shows the Embarcadero Dev-C++ 6.3 IDE with the file 'os14.cpp' open. The code implements a producer-consumer synchronization problem using C++11 threads, mutex, and condition variables. The producer thread sleeps for 500 milliseconds before producing an item, while the consumer thread waits for the condition to be met. The program is compiled and executed successfully.

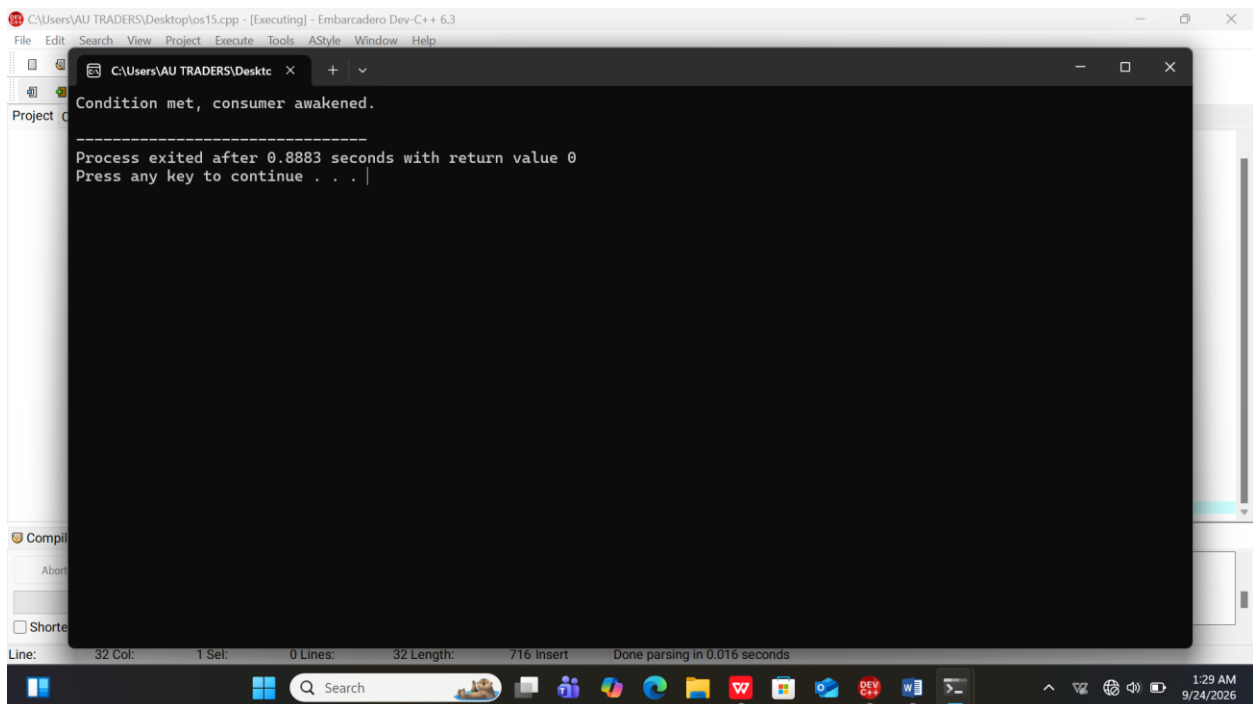
```
1  #include <thread>
2  #include <mutex>
3  #include <condition_variable>
4  #include <chrono>
5  #include <condition_variable> cv;
6  std::mutex m;
7  bool ready = false;
8
9
10 void consumer() {
11     std::unique_lock<std::mutex> lock(m);
12     // Automatically unlocks mutex and sleeps until producer notifies
13     cv.wait(lock, [] { return ready; });
14     std::cout << "Condition met, consumer awakened.\n";
15 }
16
17 void producer() {
18     std::this_thread::sleep_for(std::chrono::milliseconds(500));
19     {
20         std::lock_guard<std::mutex> lock(m);
21         ready = true;
22     }
23     cv.notify_one(); // Wake up the sleeping consumer thread
24 }
25
26 int main() {
27     std::thread tCons(consumer);
28     std::thread tProd(producer);
29     tCons.join();
30     tProd.join();
31     return 0;
32 }
```

Compiler (2) Resources Compile Log Debug Find Results Console Close

Output Filename: C:\Users\AU TRADERS\Desktop\os14.exe  
Output Size: 2.98939895629883 MiB  
Compilation Time: 2.52s

Line: 32 Col: 1 Sel: 0 Lines: 32 Length: 716 Insert Done parsing in 0.031 seconds

## Output:

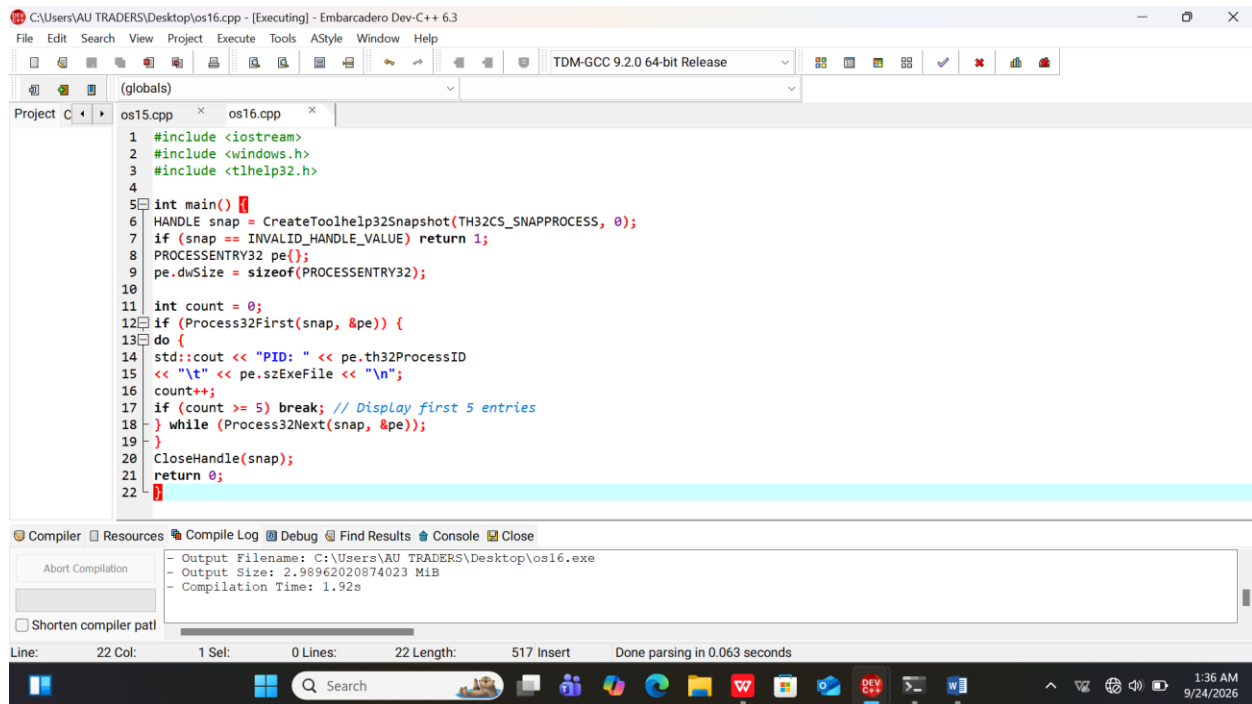


The screenshot shows the output window of the Embarcadero Dev-C++ 6.3 IDE. The output displays the message 'Condition met, consumer awakened.' followed by a separator line and the message 'Process exited after 0.8883 seconds with return value 0'. The program is executed successfully.

```
Condition met, consumer awakened.
-----
Process exited after 0.8883 seconds with return value 0
Press any key to continue . . .
```

Line: 32 Col: 1 Sel: 0 Lines: 32 Length: 716 Insert Done parsing in 0.016 seconds

## Code 16:



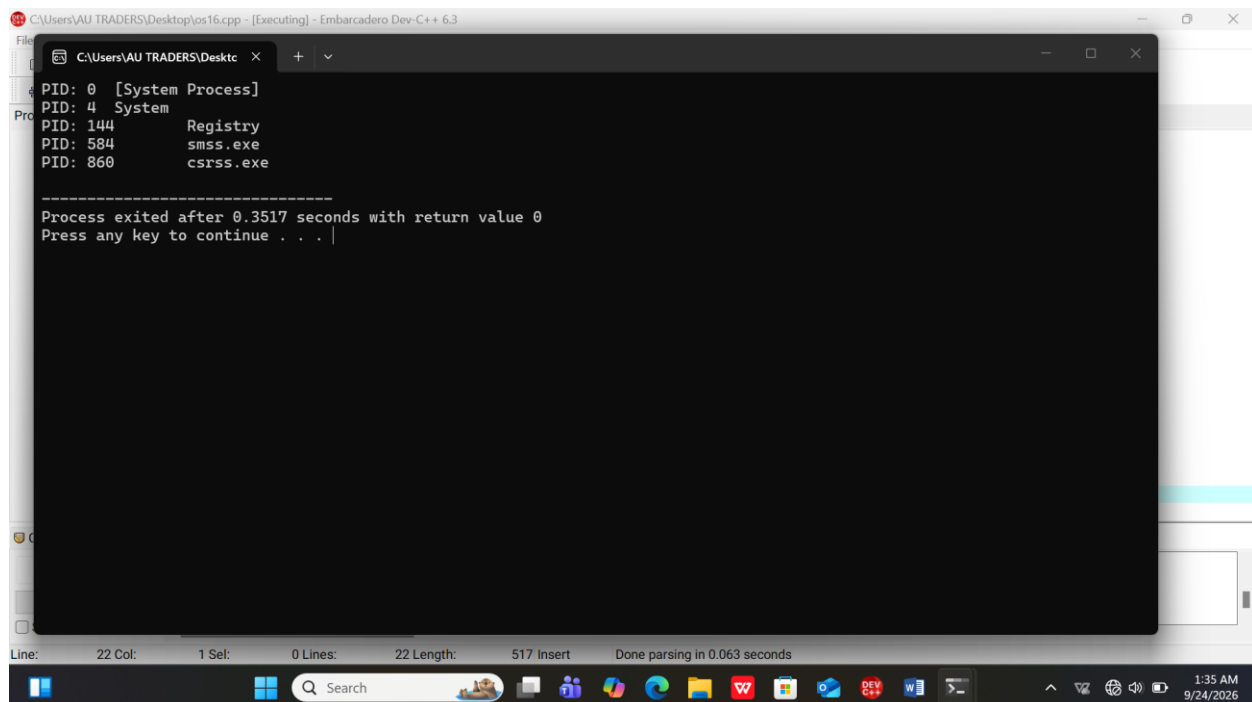
The screenshot shows the Embarcadero Dev-C++ 6.3 IDE. The main window displays the source code for `os16.cpp`. The code includes `<iostream>`, `<windows.h>`, and `<tlhelp32.h>`. It defines a `main` function that creates a snapshot of system processes using `CreateToolhelp32Snapshot` and iterates through them using `Process32First` and `Process32Next`, printing the PID and executable name of the first five processes. The compiler output window at the bottom shows the compilation details for `os16.exe`.

```
1 #include <iostream>
2 #include <windows.h>
3 #include <tlhelp32.h>
4
5 int main() {
6     HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
7     if (snap == INVALID_HANDLE_VALUE) return 1;
8     PROCESSENTRY32 pe{};
9     pe.dwSize = sizeof(PROCESSENTRY32);
10
11     int count = 0;
12     if (Process32First(snap, &pe)) {
13         do {
14             std::cout << "PID: " << pe.th32ProcessID
15                 << "\t" << pe.szExeFile << "\n";
16             count++;
17             if (count >= 5) break; // Display first 5 entries
18         } while (Process32Next(snap, &pe));
19     }
20     CloseHandle(snap);
21     return 0;
22 }
```

Compiler Output:

```
- Output Filename: C:\Users\AU TRADERS\Desktop\os16.exe
- Output Size: 2.98962020874023 MiB
- Compilation Time: 1.92s
```

## Output:

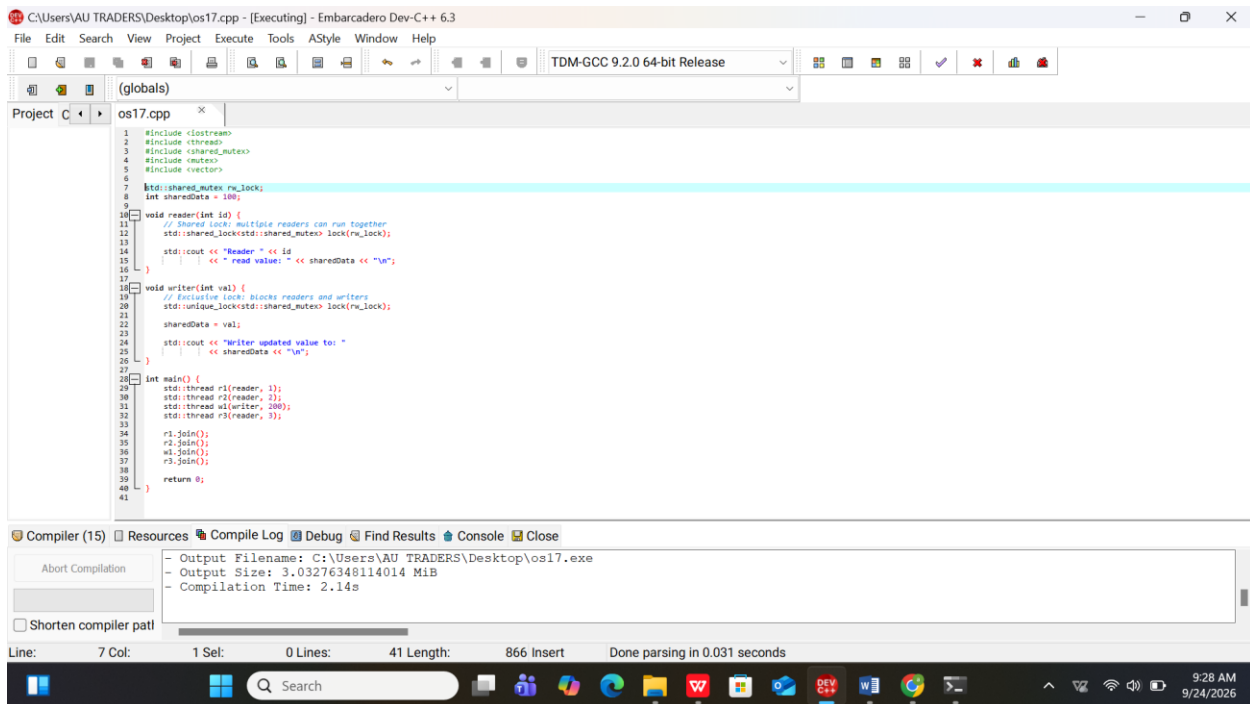


The screenshot shows the console output of the program. It lists the first five processes found in the system snapshot: PID 0 (System Process), PID 4 (System), PID 144 (Registry), PID 584 (smss.exe), and PID 860 (csrss.exe). Below this, it indicates that the process exited after 0.3517 seconds with a return value of 0.

```
PID: 0 [System Process]
PID: 4 System
PID: 144 Registry
PID: 584 smss.exe
PID: 860 csrcs.exe

=====
Process exited after 0.3517 seconds with return value 0
Press any key to continue . . .
```

## Code 18:



```
1 #include <iostream>
2 #include <thread>
3 #include <shared_mutex>
4 #include <mutex>
5 #include <vector>
6
7 std::shared_mutex rw_lock;
8 int sharedData = 100;
9
10 void reader(int id) {
11     // Shared lock: Multiple readers can run together
12     std::shared_lock<std::shared_mutex> lock(rw_lock);
13     std::cout << "Reader " << id
14               << " read value: " << sharedData << "\n";
15 }
16
17 void writer(int val) {
18     // Exclusive lock: Blocks readers and writers
19     std::unique_lock<std::shared_mutex> lock(rw_lock);
20     sharedData = val;
21     std::cout << "Writer updated value to: "
22               << sharedData << "\n";
23 }
24
25 int main() {
26     std::thread r1(reader, 1);
27     std::thread r2(reader, 2);
28     std::thread w1(writer, 200);
29     std::thread r3(reader, 3);
30
31     r1.join();
32     r2.join();
33     w1.join();
34     r3.join();
35
36     return 0;
37 }
```

Compiler (15) Resources Compile Log Debug Find Results Console Close

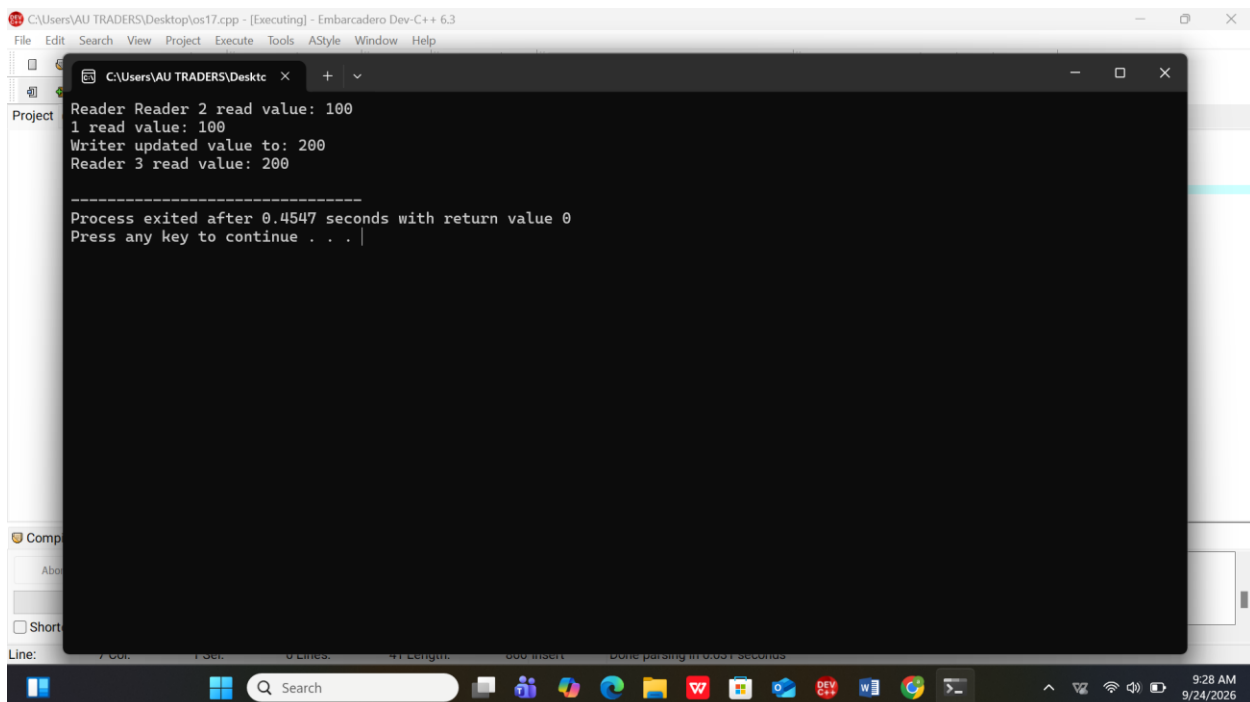
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os17.exe  
Output Size: 3.03276348114014 MiB  
Compilation Time: 2.14s

Line: 7 Col: 1 Sel: 0 Lines: 41 Length: 866 Insert Done parsing in 0.031 seconds

## Output:

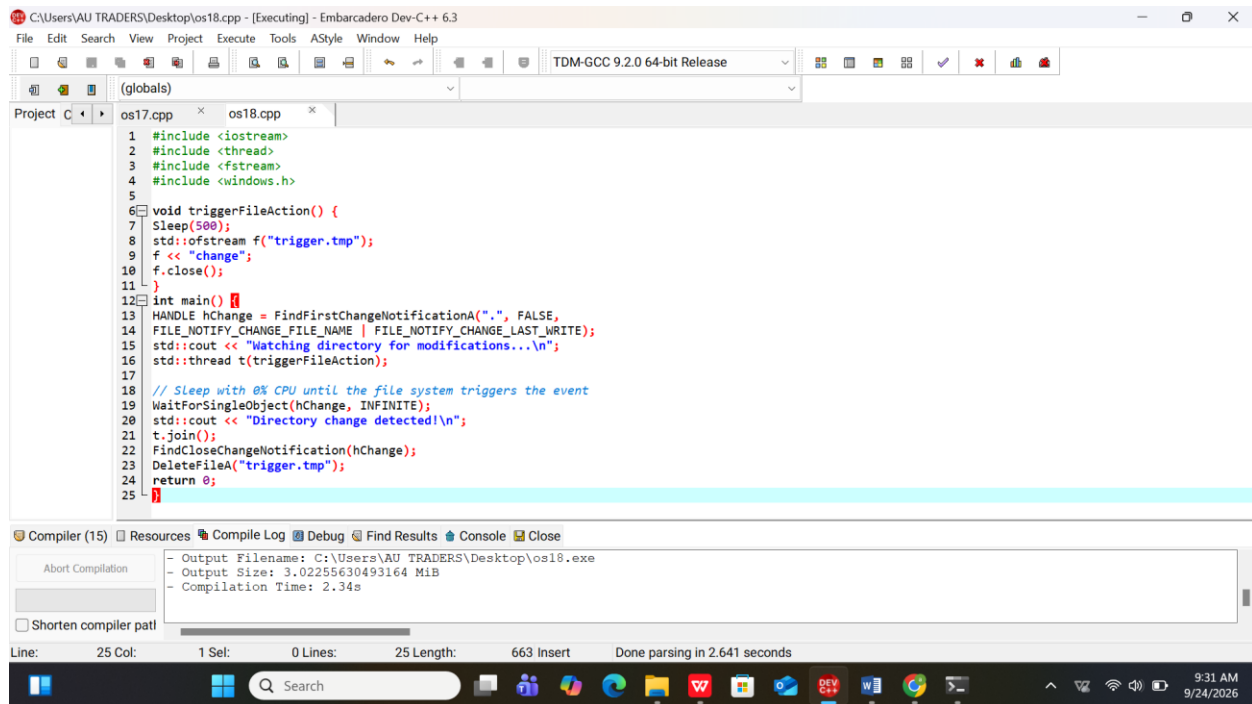


```
Reader Reader 2 read value: 100
1 read value: 100
Writer updated value to: 200
Reader 3 read value: 200

-----
Process exited after 0.4547 seconds with return value 0
Press any key to continue . . .
```

Line: 7 Col: 1 Sel: 0 Lines: 41 Length: 866 Insert Done parsing in 0.031 seconds

## Code 19:



```
1 #include <iostream>
2 #include <thread>
3 #include <fstream>
4 #include <windows.h>
5
6 void triggerFileAction() {
7     Sleep(500);
8     std::ofstream f("trigger.tmp");
9     f << "change";
10    f.close();
11 }
12
13 int main() {
14     HANDLE hChange = FindFirstChangeNotificationA(".", FALSE,
15     FILE_NOTIFY_CHANGE_FILE_NAME | FILE_NOTIFY_CHANGE_LAST_WRITE);
16     std::cout << "Watching directory for modifications...\n";
17     std::thread t(triggerFileAction);
18
19     // Sleep with 0% CPU until the file system triggers the event
20     WaitForSingleObject(hChange, INFINITE);
21     std::cout << "Directory change detected!\n";
22     t.join();
23     FindCloseChangeNotification(hChange);
24     DeleteFileA("trigger.tmp");
25     return 0;
26 }
```

Compiler (15) Resources Compile Log Debug Find Results Console Close

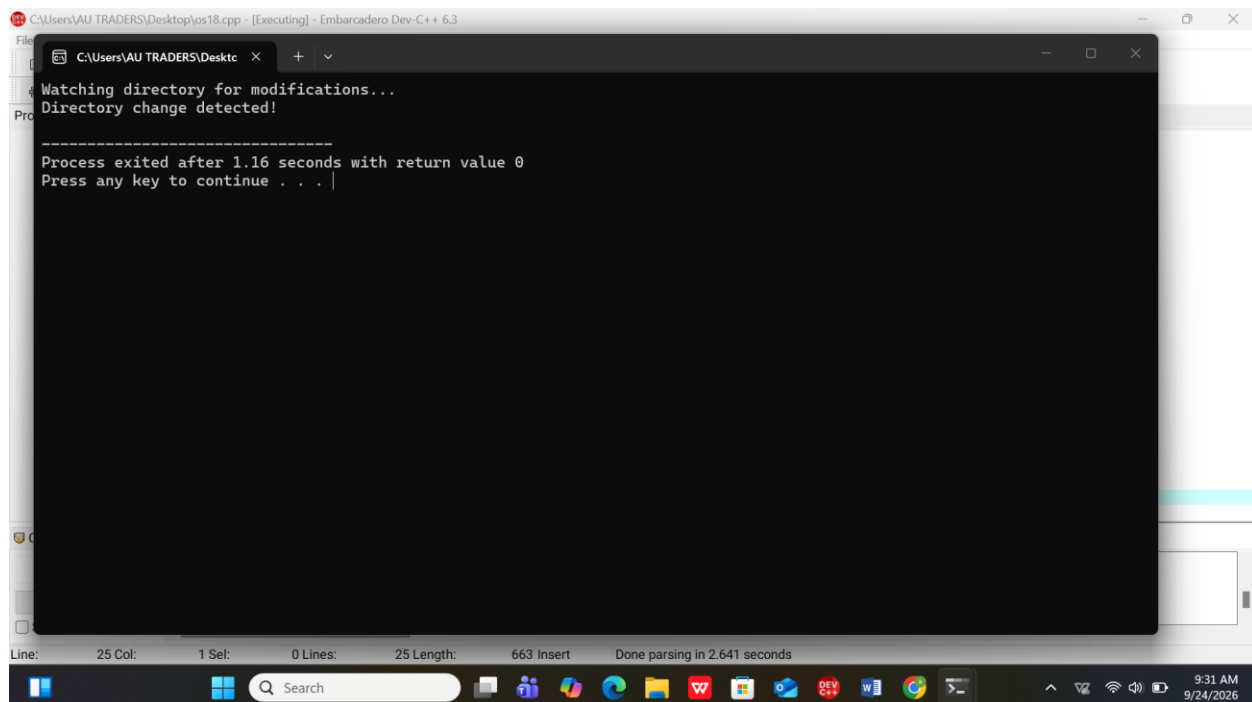
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os18.exe  
Output Size: 3.02255630493164 MiB  
Compilation Time: 2.34s

Line: 25 Col: 1 Sel: 0 Lines: 25 Length: 663 Insert Done parsing in 2.641 seconds

## Output:

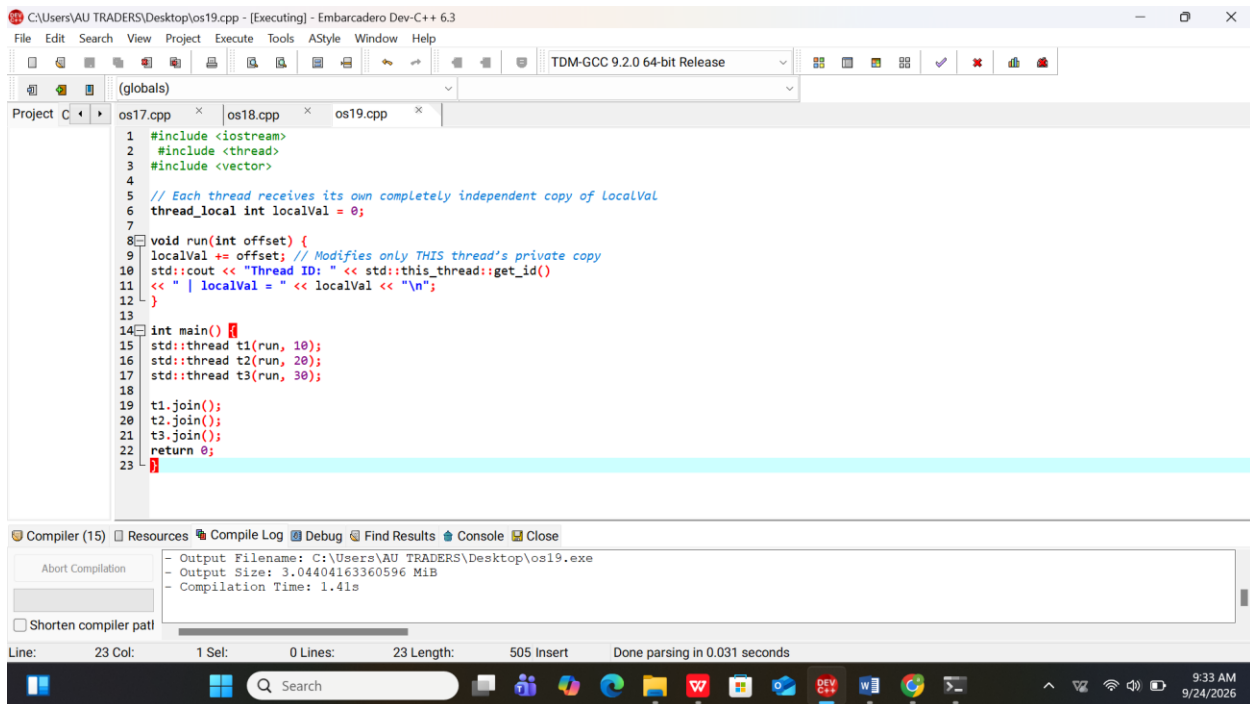


```
C:\Users\AU TRADERS\Desktop\os18.exe
Watching directory for modifications...
Directory change detected!

-----
Process exited after 1.16 seconds with return value 0
Press any key to continue . . .
```

Line: 25 Col: 1 Sel: 0 Lines: 25 Length: 663 Insert Done parsing in 2.641 seconds

## Code 20:



```
1 #include <iostream>
2 #include <thread>
3 #include <vector>
4
5 // Each thread receives its own completely independent copy of localVal
6 thread_local int localVal = 0;
7
8 void run(int offset) {
9     localVal += offset; // Modifies only THIS thread's private copy
10    std::cout << "Thread ID: " << std::this_thread::get_id()
11    << " | localVal = " << localVal << "\n";
12 }
13
14 int main() {
15     std::thread t1(run, 10);
16     std::thread t2(run, 20);
17     std::thread t3(run, 30);
18
19     t1.join();
20     t2.join();
21     t3.join();
22     return 0;
23 }
```

Compiler (15) Resources Compile Log Debug Find Results Console Close

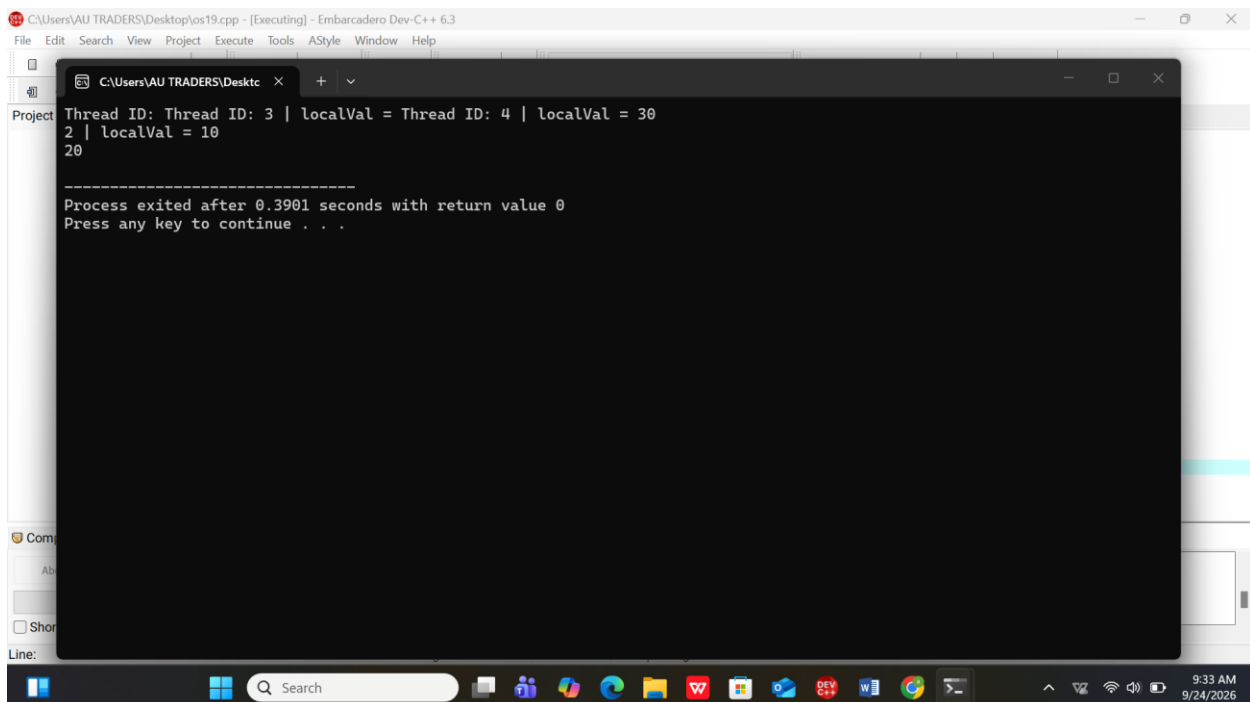
Abort Compilation

Shorten compiler path

Output Filename: C:\Users\AU TRADERS\Desktop\os19.exe  
Output Size: 3.04404163360596 MiB  
Compilation Time: 1.41s

Line: 23 Col: 1 Sel: 0 Lines: 23 Length: 505 Insert Done parsing in 0.031 seconds

## Output:



```
Thread ID: Thread ID: 3 | localVal = Thread ID: 4 | localVal = 30
2 | localVal = 10
20
-----
Process exited after 0.3901 seconds with return value 0
Press any key to continue . . .
```